ravindu439: On-Chip Neuromorphic Accelerator — Project Summary

1. What We Are Building An on-chip offline learning SNN (Spiking Neural Network) accelerator implemented as a custom RISC-V SoC peripheral.

The system has 3 operating states:

State Who Runs It What Happens INFERENCE Neuromorphic Accelerator Processes spike packets, computes membrane potentials (V_mem) SURROGATE SUBSTITUTION General-purpose CPU Reads V_mem from shared memory, queries Surrogate LUT over Wishbone bus, writes surrogate gradients back LEARNING RISC-V Processor Reads surrogate gradients from shared memory, runs backpropagation, updates weights The goal of this session was to implement and verify the backpropagation data path — specifically ensuring the pre-fire membrane potential (v_pre_spike) is correctly captured and written to shared memory after each timestep, so the CPU can compute surrogate gradients.

2. High-Level Architecture External spikes │ ▼ Main FIFO (11-bit spike packet: {cluster_id[5:0], neuron_id[4:0]}) │ ▼ spike_forwarder_8 (SF8) — routes to 8 groups of 4 clusters │ ▼ spike_forwarder_4 (SF4) — routes to individual clusters │ ▼ neuron_cluster[0..31] ├── incoming_forwarder (accepts/filters spikes by source cluster) ├── weight_resolver (looks up synaptic weight rows by source cluster_id) ├── neuron_layer (32 neurons, each with a potential_adder) │ └── potential_adder (accumulates weights, threshold comparison, spike/reset) ├── v_pre_spike_out [32×32-bit bus] ← NEW (for backprop) ├── v_mem_out [32×32-bit bus] (post-fire, for debug) └── spikes_out_raw [32-bit mask] │ ▼ (wide parallel bus — NOT through NoC) neuron_accelerator.v ├── all_pre_spike [1024×32-bit] ← collected from all 32 clusters ├── all_spikes [1024-bit] └── Dump FSM: DUMP_IDLE → DUMP_VMEM (1024 cycles) → DUMP_SPIKES (32 cycles) → DUMP_DONE ↓ ↓ portb_addr/portb_din/portb_we → Shared BRAM (Port B) │ ▼ Shared Memory (BRAM, Port A = Wishbone–CPU, Port B = Accelerator dump) │ ▼ RISC-V CPU reads via Wishbone → computes h'(v_pre_spike) from Surrogate LUT → backpropagation Key Design Decision: Two Separate Data Paths Spike NoC (SF8→SF4): Only routes 11-bit spike packets between clusters during inference Direct dump bus: v_pre_spike_out wires from all clusters → all_pre_spike → Dump FSM → BRAM (no NoC involved)
3. Repository Structure inference_accelarator/ ├── neuron_accelerator/ │ ├── neuron_accelerator.v ← TOP MODULE (modified) │ ├── neuron_accelerator_tb.v ← existing full XOR testbench │ └── dump_fsm_tb.v ← NEW Level 4 testbench (in progress) ├── neuron_cluster/ │ ├── neuron_cluster.v ← modified (added v_pre_spike_out) │ ├── neuron_cluster_tb.v ← NEW Level 2 testbench (PASSES 8/8) │ └── neuron_layer/ │ └── neuron_layer.v ← modified (added v_pre_spike_out bus) ├── neuron_integer/ │ ├── neuron_int_all/ │ │ ├── adder/potential_adder.v ← modified (v_pre_spike now REGISTERED) │ │ └── neuron/neuron.v ← modified (added v_pre_spike_out port) │ └── neuron_int_lif/ │ ├── adder/potential_adder.v ← modified (v_pre_spike now REGISTERED) │ └── neuron/neuron.v ← modified (added v_pre_spike_out port) ├── spike_network/ │ ├── spike_forwarder_4.v │ ├── spike_forwarder_4_tb.v ← NEW Level 3 testbench (PASSES 5/5) │ └── spike_forwarder/spike_forwarder.v ├── weight_resolver/weight_resolver.v └── initialization_router/self_data_mng.v
4. Files Modified — What and Why neuron_int_all/adder/potential_adder.v ☆ CRITICAL FIX What changed: v_pre_spike converted from output wire (combinational) to output reg (registered).

Why this was critical:

verilog ORE (BUG): combinational — only valid 1 clock cycle assign v_pre_spike = weight_added[31:0]; ER (FIX): registered — holds value until next timestep output reg [31:0] v_pre_spike, port declaration ... ide always @(posedge clk): end else if(adder_state == 2) begin putation cycle spike <= (weight_added > v_threshold); final_potential <= (weight_added > v_threshold) ? 0 : weight_added; v_pre_spike <= weight_added[31:0]; atched here, holds value done <= 1; end re_spike now behaves like final_potential: holds value until next computation weight_added is a combinational expression of input_weight (from weight_resolver) + decayed_potential. The weight_resolver only pulses weights_in for 1 clock cycle. The dump FSM needs 1024+ cycles to write everything. Without the latch, the dump FSM would read zeros after cycle 1.

Root cause of why Level 2 test worked but Level 4 didn't: In neuron_cluster_tb.v , weights_in was driven by a testbench reg held at a constant value, so weight_added stayed valid indefinitely. In the full accelerator, weights_in comes from weight_resolver which pulses for only 1 cycle.

neuron_int_lif/adder/potential_adder.v Same fix as above but for the LIF model (simpler FSM — computation happens at adder_state==1).

neuron_int_all/neuron/neuron.v Added v_pre_spike_out output port and wired it to potential_adder's v_pre_spike.

neuron_int_lif/neuron/neuron.v Same as above for LIF neuron. Also updated potential_adder instantiation to connect new v_pre_spike port.

neuron_layer/neuron_layer.v Added v_pre_spike_out [32*NEURONS-1:0] output bus. Generates connections:

verilog assign v_pre_spike_out[i*32 +: 32] = neuron_inst[i].v_pre_spike_out; neuron_cluster/neuron_cluster.v Added v_pre_spike_out [32*neurons_per_cluster-1:0] output port, connected from neuron_layer_inst.v_pre_spike_out.

neuron_accelerator/neuron_accelerator.v Three changes:

Added wire [32*N_TOTAL-1:0] all_pre_spike; (alongside existing all_vmem) Connected .v_pre_spike_out(all_pre_spike[(i*4+j)*32*N_PER_CLUSTER +: 32*N_PER_CLUSTER]) in the cluster generate loop Dump FSM now writes all_pre_spike[dump_idx*32 +: 32] instead of all_vmem[...] 5. Bottom-Up Test Strategy Level 2: neuron_cluster_tb.v — ✅ PASSES 8/8 Location: inference_accelarator/neuron_cluster/

Run:

bash cd /ravindu/FYP/e20-4yp-onchip-offline-neuromorphic-computing/inference_accelarator/neuron_cluster iverilog -o neuron_cluster_tb.vvp neuron_cluster_tb.v && vvp neuron_cluster_tb.vvp 2>&1 What it tests:

Configures 4 neurons (N0, N1, N3, N5) with different thresholds N0: weight=5.0 Q16.16, VT=3.0 → FIRES (spike[0]=1, v_pre=5.0) N1: weight=2.0 Q16.16, VT=3.0 → NO FIRE (spike[1]=0, v_pre=2.0) N3: weight=8.0 Q16.16, VT=6.0 → FIRES (spike[3]=1, v_pre=8.0) N5: weight=1.5 Q16.16, VT=6.0 → NO FIRE (spike[5]=0, v_pre=1.5) 8 checks: 4 spike bits + 4 pre-fire V_mem values Key insight confirmed: v_pre_spike_out shows the value before threshold reset, which is what backprop's surrogate gradient h'(v) needs. v_mem_out (post-fire) shows 0 for fired neurons — wrong for backprop.

Level 3: spike_forwarder_4_tb.v — ✅ PASSES 5/5 Location: inference_accelarator/spike_network/

Run:

bash cd /ravindu/FYP/e20-4yp-onchip-offline-neuromorphic-computing/inference_accelarator/spike_network iverilog -o spike_forwarder_4_tb.vvp spike_forwarder_4_tb.v && vvp spike_forwarder_4_tb.vvp 2>&1 What it tests:

Init routing table: forwarding_map[main] = 5'b00110 → routes to cluster 0 AND cluster 1 Test 1: Single spike {cluster=62, n=0} = 0x7C0 → both cluster 0 and cluster 1 receive it; clusters 2 and 3 stay empty Test 2: Two back-to-back spikes (n=0 then n=1) → both delivered in correct order to cluster 0 Test 3: done signal goes HIGH after all output FIFOs are drained Key insight learned: The spike network is a forwarding network only — routing based on which INPUT PORT the spike came from, not the content of the packet. The packet content {src_cluster_id, src_neuron_id} is only used by the DESTINATION cluster's incoming_forwarder and weight_resolver to look up the correct weight row.

Important bug found and fixed in testbench: spike_forwarder has a 2-cycle pipeline:

Cycle 1: sees rd_en, latches which port to read from Cycle 2: reads the data WIRE and writes to output FIFO So data must be held stable for 2 cycles after rd_en goes high, or the wrong data gets forwarded.

Level 4: dump_fsm_tb.v — 🔄 IN PROGRESS Location: inference_accelarator/neuron_accelerator/

Run:

bash cd /ravindu/FYP/e20-4yp-onchip-offline-neuromorphic-computing/inference_accelarator/neuron_accelerator iverilog -o dump_fsm_tb.vvp dump_fsm_tb.v && vvp dump_fsm_tb.vvp 2>&1 What it tests:

Uses same data_mem.mem + spike_mem.mem as full XOR testbench (correct weight initialization) Injects XOR(1,0) spike pattern (index 40 in spike_mem) — input neuron 0 fires Captures all Port B writes into a local reg array Verifies: non-zero V_mem written, correct write counts (1024 V_mem + 32 spike words), spike bits set Status: Previous versions showed all-zeros — root cause was the v_pre_spike combinational wire bug (now fixed). The fix to potential_adder.v should resolve this. Needs rerun to confirm.

Known remaining issue: Dump runs TWICE (write_count=2112) because accelerator_done pulses once after init and once after inference. The init-triggered dump writes all zeros (no inference yet). This is cosmetically wrong (doubles the write count check) but functionally harmless for the CPU. Need to either:

Filter by expected address range in write counter Or add a dump_enable gate that only opens after first time_step 6. Key Protocols and Data Formats Spike Packet Format (11-bit) [10:5] = source cluster_id (6 bits) [4:0] = source neuron_id (5 bits) Example: {6'd62, 5'd0} = 0x7C0 (external input cluster 62, neuron 0) Weight Format (Q16.16 fixed-point) Bits [31:16] = integer part Bits [15:0] = fractional part Examples: 5.0 = 0x00050000 2.0 = 0x00020000 1.5 = 0x00018000 3.0 = 0x00030000 (threshold) Init Router (self_data_mng) Byte Protocol The init data reaches each cluster through a tree router. self_data_mng splits init bytes:

Byte 0: data_in[0] = port_select (0=forwarder/cluster, 1=weight_resolver) Byte 1: count = number of bytes that follow Bytes 2..N: data forwarded to selected module Weight Resolver Init Protocol Byte 0: weight_addr[7:0] (row address low byte) Byte 1: weight_addr[15:8] (row address high byte) Byte 2: flit_count (number of weight bytes = N_NEURONS * 4) Bytes 3..N: weight data bytes (packed, 32-bit per neuron) Dump FSM Memory Layout V_mem region: addr = vmem_base + (timestep × N_TOTAL) + neuron_index Spike region: addr = spike_base + (timestep × N_SPIKE_W) + word_index Default config: vmem_base = 0xA000,

spike_base = 0xB000 N_TOTAL = 32 clusters × 32 neurons = 1024 N_SPIKE_W = 1024 32 = 32 words Neuron mapping: neuron_index = (cluster_group × 4 + cluster_within_group) × 32 + neuron_within_cluster 7. Important Bugs Found and Fixed Bug Location Root Cause Fix v_pre_spike always zero in full accelerator potential_adder.v (both versions) Was a combinational wire of weight_added, which is only valid for 1 clock cycle when weight_resolver pulses. Dump FSM reads 1024 cycles later. Changed to output reg, latched at computation time (adder_state==2/1) alongside final_potential Level 2 test got zeros initially neuron_cluster_tb.v Used wrong opcode structure for init packets Fixed init packet sequence Level 3 test timed out for back-to-back spikes spike_forwarder_4_tb.v Used @(posedge main_rd_en) — if signal stays high between 2 spikes, no rising edge detected Changed to wait(main_rd_en == 1'b1) and held data for 2 cycles (pipeline depth) Level 3 done never went high spike_forwarder_4_tb.v Output FIFOs weren't drained before checking done. done = &fifo_empty_out && ... Added drain loop before done check XOR(0,0) produced all-zero V_mem (thought to be a bug) dump_fsm_tb.v NOT a bug — XOR(0,0) has no input spikes → no weights loaded → V_pre = 0 is correct Changed to use XOR(1,0) pattern (spike_mem offset 40) 8. Current Status Level Module Test File Status L2 neuron_cluster neuron_cluster_tb.v ✅ 8/8 PASS L3 spike_forwarder_4 spike_forwarder_4_tb.v ✅ 5/5 PASS L4 neuron_accelerator dump FSM dump_fsm_tb.v 🔄 Root cause fixed, needs rerun L5 Full integration (XOR) neuron_accelerator_tb.v ⏳ Not yet 9. Next Steps Rerun dump_fsm_tb.v — confirm non-zero V_mem now that v_pre_spike is registered Fix double-dump issue in dump_fsm_tb.v — the dump runs once after init (all zeros) then once after inference. Fix the write_count check to only count the second dump Level 5: Full integration — run neuron_accelerator_tb.v with XOR data, verify CPU readback via Wishbone gives correct spike outputs Surrogate LUT integration — once dump is verified working, integrate the Wishbone surrogate LUT so the CPU can read V_mem and query the LUT RISC-V C code for backprop — write the learning phase firmware that reads V_mem, queries surrogate LUT, and performs weight update 10. Environment Simulator: iverilog + vvp Language: Verilog (.v) Number format: Q16.16 fixed-point for membrane potentials and weights Cluster config for XOR: cluster_group_count=8, number_of_clusters=32, neurons_per_cluster=32 External input cluster ID: 62 (0x3E) — used for virtual input spikes in the XOR test

here is current project summary on this project currently im testing out the accelarator dumping part it has some issues . so before moving on please analyse my code base for better understand .and then i am trying to do is since this works ffor the neuron cluster test bench instead of relaying the data mem and spike mem files how about if we use the same data and same values according to the packes and heirachy and then we can actually verfy the out put and the debug the issue since we know the expected out put right . as first step analyse the code base

GitHub Copilot:

I'll analyze your codebase systematically, starting from the bottom of the hierarchy and working up to the neuron accelerator level. Let me examine the key files. Now let me look at the dump FSM testbench and the neuron cluster testbench: Now let me look at the LIF neuron variant (since neuron_accelerator.v includes the LIF version), and key supporting modules: Now let me look at the weight resolver, incoming_forwarder, spike forwarder, and init router: Now let me look at the incoming_forwarder, cluster_controller, spike_forwarder_4, and the full testbench: Now let me look at the neuron_accelerator_tb.v (the full XOR testbench) and the generate_xor_test.py to understand the data_mem.mem and spike_mem.mem formats: Now let me also look at the remaining part of the full XOR testbench to understand the inference loop: Now let me check the spike_forwarder_4 and spike_forwarder_8 to understand the routing: Now let me also look at the LIF encoding and accumulator:

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot:

Let me continue the analysis by examining the remaining key files I haven't looked at yet: Now let me check if data_mem.mem and spike_mem.mem exist and look at what the LIF controller expects: Now let me check the LIF neuron's potential_adder.v more carefully — the neuron_accelerator.v includes the LIF variant, not the all variant. Let me also check the init_router: Now let me look at the `gen_simple_test_mem.py` file: Now I have a comprehensive understanding of the codebase. Let me also quickly check the outgoing_enc module: Now I have a thorough understanding of the entire codebase. Here's my complete analysis:

Complete Codebase Analysis

1. Architecture Overview — Data Flow

The system has **two distinct data paths**, which is crucial to understand:

Path A: Spike NoC (Inference)

```
External spike → main_fifo_in → spike_forwarder_8 →
spike_forwarder_4[group] → cluster input FIFO
  → incoming_forwarder (matches src cluster_id → weight BRAM row)
  → weight_resolver (reads BRAM row → 32×32-bit weights_in bus)
  → accumulator (accumulates weights_in per neuron over multiple spikes)
  → [time_step edge] → potential_decay → potential_adder (accumulated +
decayed → weight_added)
  → spike decision + v_pre_spike latch
  → outgoing_enc → cluster output FIFO → spike_forwarder_4 →
spike_forwarder_8 → main_fifo_out
```

Path B: Dump Bus (Backprop readout)

```
all 32 clusters' v_pre_spike_out wires (32×32 bits each = 1024×32 bits
total)
  → all_pre_spike[32767:0] in neuron_accelerator.v
  → Dump FSM: DUMP_IDLE → DUMP_VMEM (1024 writes) → DUMP_SPIKES (32 writes)
→ DUMP_DONE
  → portb_addr/portb_din/portb_we → Shared BRAM Port B
```

2. Key Module Interactions

neuron_accelerator.v — the top module

- **Includes LIF variant** (`neuron_int_lif`), NOT `neuron_int_all`. This is important: the LIF `potential_adder` has a simpler FSM (`adder_state` is 1-bit, computation at `adder_state==1`).
- **32 clusters** organized as 8 groups × 4 clusters.
- **Dump FSM trigger**: `accelerator_done && !prev_clusters_done` (rising edge of `accelerator_done`).
- `accelerator_done = all_clusters_done & all_forwarders_done & main_fifo_empty_in & main_fifo_empty_out & resolvers_done`.

weight_resolver.v — critical data flow

- Shared across 4 clusters per group. Uses a **priority-selected FIFO buffer** — one cluster at a time gets its weight row served.
- `load_weight_out[i]` pulses for **exactly 1 clock cycle** per weight row. This means `weights_in` at the cluster is valid for only 1 cycle.
- The `accumulator` latches this on `load` signal (which is `load_weight_in` in inference mode).
- The accumulated value is then fed to `potential_adder.input_weight` on the time_step edge.

Accumulator behavior (crucial timing)

- During inference (`chip_mode=0`), `acc_load = load_data` which is wired to `load_weight_in` from the weight_resolver.
- `accumulated_out` is registered: it only updates on the `time_step` rising edge, outputting the sum from the previous period.
- So `input_weight` to `potential_adder` = `accumulated_out` = sum of all weights loaded since last time_step.

LIF `potential_adder` data path

```
weight_added = decayed_potential + input_weight  (combinational)
```

- At `adder_state==1`: `v_pre_spike <= weight_added` (REGISTERED — the fix).
- `weight_added` depends on `input_weight` = `accumulated_out`, which holds its value until next time_step. So `weight_added` is **stable** during the dump window ✅.
- Wait — actually, looking more carefully at the LIF variant: `decayed_potential = output_potential_decay` which is also registered, so `weight_added` should indeed be stable.

3. Critical Issue Found: The `v_pre_spike` Registration

In the LIF potential_adder.v (the one actually used by neuron_accelerator.v):

```
assign weight_added = decayed_potential + input_weight;  // combinational
// ...
end else if(adder_state == 1) begin
    v_pre_spike <= weight_added; // latch pre-fire value
```

This is **correctly registered** — `v_pre_spike` holds its value until the next computation. Since `input_weight` comes from `accumulated_out` (registered) and `decayed_potential` comes from `output_potential_decay` (registered), `weight_added` is actually stable even as a combinational wire. But the registration of `v_pre_spike` adds an extra safety layer.

4. The Double-Dump Issue

Looking at the dump FSM trigger:

```
if (accelerator_done && !prev_clusters_done) begin  // rising edge
```

After init completes, `accelerator_done` goes high (all FIFOs empty, all clusters done). This triggers the **first dump** — which writes all zeros (no inference happened yet). Then when time_step is asserted and inference completes, `accelerator_done` rises again, triggering the **second dump** with real data.

In dump_fsm_tb.v, the `write_count` check expects exactly `N_TOTAL + N_SPIKE_W = 1056`, but will get `2112` due to the double dump.

5. Neuron Cluster TB vs Full Accelerator — The Key Difference

In neuron_cluster_tb.v:

- `weights_in` is a **testbench reg held constant** → `weight_added` stays valid indefinitely.
- `load_weight_in` is asserted manually alongside `time_step`.
- No weight_resolver, no spike NoC, no init_router.

In the full accelerator:

- `weights_in` comes from `weight_resolver` which pulses for 1 cycle.
- The `accumulator` captures it and holds `accumulated_out` until `time_step`.
- `input_weight = accumulated_out` is stable at computation time.
- The `v_pre_spike` registration ensures the value holds through the dump window.

6. Your Proposed Approach — Analysis

Your idea is sound: **instead of relying on `data_mem.mem`/`spike_mem.mem` (opaque binary blobs), create a dump_fsm test that uses the same known values as neuron_cluster_tb.v**, so you can predict exact outputs. This eliminates unknowns.

The challenge is that at the accelerator level, data must flow through:

1. **init_router** (upper → lower → self_data_mng → cluster_controller/weight_resolver)
2. **spike_forwarder_8** → **spike_forwarder_4** (routing table must be initialized)
3. **weight_resolver** (BRAM must be loaded via init protocol)
4. **incoming_forwarder** (cluster_id table and base address must be configured)

So you can't just set `weights_in` directly — you need to generate the correct init byte stream.

7. Recommended Strategy

Create a **minimal accelerator testbench** that:

1. Uses generate_xor_test.py's packet builder functions to generate init bytes for a **single known neuron** (e.g., cluster 0, neuron 0, weight=5.0, VT=3.0 → should fire, v_pre_spike=5.0).
2. Injects a single spike from external cluster 62.
3. Asserts `time_step`.
4. Captures Port B writes.
5. Verifies `portb_din` at the expected neuron index matches `0x00050000`.

This gives you **end-to-end traceability** with a known expected output, just like the L2 test but going through the full init_router → weight_resolver → neuron → dump FSM pipeline.

Would you like me to create this minimal dump FSM testbench now?

ravindu439: yes i like for that idea so you mean since we use registers in the neuron cluster test bench we passed the test right .and what changes so then we need in the accelarator top level to propagate this v mem and spike data in to theshared memory . and you said that happen since it use single clock cycle . so can i see the difference using gtkwave so i can visually see the issue in here

GitHub Copilot: Great question! Let me clarify the exact issue and then build a self-contained testbench with known values that you can visualize in GTKWave.

Let me first run the current dump_fsm_tb to see what it actually outputs now: Good — so we can confirm: double-dump (2112 writes) and all V_mem = 0. Now let me check the neuron cluster tb to confirm it passes: Good — L2 confirms 8/8. Now I understand exactly what needs to be done. Let me build a **self-contained accelerator testbench** that:

1. Uses the **same exact values as L2** (neuron 0: weight=5.0, VT=3.0 → fires, v_pre=5.0)
2. Goes through the **real init_router** → **weight_resolver** → **spike_forwarder** chain
3. Has **rich GTKWave-friendly** `$dumpvars` so you can visually trace every stage
4. Has the **double-dump fix** built in This is a critical insight! Now I see the exact timing:

- On `time_step` rising edge: `accumulated_out <= accumulated_reg` (snapshot of sum)
- Then `accumulated_reg <= 0` (reset for next period)
- So `potential_adder` sees `input_weight = accumulated_out` only **after** `time_step` edge

Now I have everything I need. Let me build the complete testbench with embedded known init bytes and rich GTKWave signals:

Made changes.

The neuron_accelerator.v defines `NEURON_ACCELERATOR` guard at top — need to check how it's included: The issue is `integer` declarations inside unnamed `begin` blocks. Let me fix those to named blocks and the `reg sw0` declaration issue:

```
Made changes.
```

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot:

```
Made changes.
```

Let me continue compiling and fixing the testbench: Good — only warnings, no errors. Let me check if there's a top-level module issue: It compiled successfully. Now run it: Good progress. The first dump completes (132 writes = 128+4 ✓) but then `dump_done` never goes high after the first dump. The issue is that the first dump finishes (sets `dump_done`) but the testbench never catches it because the order of events is wrong — the dump actually happens BEFORE `init_done` is set. Let me fix the timing logic:

```
Made changes.
```

The scaffold is working perfectly. Now we can clearly see the real bug: **the spike is injected but the weights are not reaching the neurons** — all V_mem = 0x00000000 even in dump 1 (post-inference). This means the issue is in the spike routing + weight_resolver chain. The spike was injected but the `time_step` is asserted too soon (after only 30 cycles, before the spike propagates). Let me check the timing and also add diagnostic prints to see where the spike gets lost:

```
Made changes.
```

The timing is fine. The issue is that weights never reach neurons — `accelerator_done` goes high quickly after the spike but with no weight accumulation. Now let's look at the VCD with GTKWave to trace exactly where the spike dies. First let me add more targeted diagnostic prints directly to the TB, focused on the spike flow:

```
Made changes.
```

Excellent — this is a very revealing trace. **The spike packet is `0x000` at every stage instead of `0x7C0`**. That means the `spike_forwarder` is reading the data at the wrong time — it's reading **zeros from the FIFO** even though `0x7C0` was written. This is the classic 2-cycle pipeline issue in spike_forwarder.v. The packet `0x7C0` was written to main_fifo but by the time SF8 reads it, it reads `0x000`.

Looking at spike_forwarder.v: it has a **2-stage pipeline**: Stage 1 captures `current_selected_port_index`, Stage 2 actually reads the data wire. The FIFO `dout` is a registered output — by the time the forwarder's stage 2 reads it, the FIFO has already moved on because `rd_en` was asserted and the data presented is the *next* word.

The real issue: **the main FIFO in the accelerator has its own data latched**, but SF8 reads data off `main_fifo_dout_in` which is a wire that updates combinatorially with `rd_en`. The spike data `0x7C0` was in the FIFO, SF8 asserted `rd_en`, and in the second pipeline stage the data was correctly `0x7C0` — but the FIFO is not a simple register, so when SF8 then tries to write it forward, it writes what it sees at that moment. Let me check the FIFO module: Found it. **The FIFO outputs zero when not reading**: `else begin dout <= 0; end`. This means `dout` is `0` in every cycle except the cycle immediately after `rd_en` is asserted. The `spike_forwarder`'s **2-stage pipeline** reads `main_out_data_out` in Stage 2 (one cycle after asserting `rd_en`) — and at that point the FIFO has already updated `dout` to the data. But looking at the trace: `data=0x000` at SF8 read.

The problem is the **FIFO dout behavior**: `dout <= mem[rd_ptr]` is registered — it shows the data *one cycle after* `rd_en`. But spike_forwarder.v Stage 2 reads `main_out_data_out` in the same cycle as `store_selected_port`. Let me check the spike_forwarder pipeline timing again — specifically, when it reads `main_out_data_out`:

In spike_forwarder.v Stage 2 (`store_selected_port > -1`): it reads `main_out_data_out` directly. But `main_out_data_out` is `dout` from the FIFO — registered, updated only on `rd_en` cycle. Since `store_selected_port` is set in cycle N+1 after `selected_port` in cycle N asserted `rd_en`, the FIFO `dout` has the data available at cycle N+1. That should work.

The real trace issue is `data=0x000` — the spike packet is `0x7C0` but the SF8 is reading `0x000`. This means the SF8 forwarding map row 0 is **not routing to group 0** — the init bytes for SF8 are not being parsed correctly. The `0x7C0` packet enters main_fifo, SF8 reads it (`rd_en` asserted), but forwards to nobody (map[0]=0), and discards it. Let me verify by checking the SF8 controller format more carefully: Now I see the issue clearly. The `spike_forwarder_controller_8` has a critical behavior: **when `load_data` goes low, it resets `forwarding_row <= 0`**. This means it zeros out the forwarding_row between the two bytes. But

more importantly, in spike_forwarder.v the `load_row` only stays high for 1 cycle — that's fine, the map gets written.

The real issue is the **`else begin load_row <= 0; forwarding_row <= 0; end`** block — it clears `forwarding_row` every idle cycle. But the map is already written into `forwarding_map[]` in spike_forwarder.v before this happens, so that's OK.

The deeper issue: looking at spike_forwarder.v line `else begin dout <= 0; end` in the FIFO — when SF8 reads from main_fifo (asserts `rd_en`), in the NEXT cycle `dout` = `0x7C0`. SF8's Stage 1 stores `store_selected_port`, Stage 2 uses `main_out_data_out`. But the FIFO cleared `dout` to 0 in the cycle AFTER the read because there are no more items — **`dout <= 0` when `do_read` is false**. Since Stage 2 runs 1 cycle after `rd_en`, it reads `main_out_data_out` = the new `dout` = `0x7C0`. That should be correct.

The `data=0x000` in the trace means the forwarding_map[0] for SF8 has **bit 1 = 0**, so SF8 is not forwarding to group 0. The init bytes for SF8 are not reaching the controller. The issue is in how the `init_router` routes to the upper self (`0xA0` address). Let me check what `ROUTER_ID` and routing logic means:

Looking at neuron_accelerator.v: `init_router_upper` has `ROUTER_ID(8'b10100000)` = `0xA0`. When a packet comes in with `header_flit == ROUTER_ID` (`0xA0`), it routes to `this_router` (the spike_forwarder_8 controller). The **second flit** is the `count`, then the remaining bytes go to `load_data`/`data` of the SF8 controller.

In the init_rom: `[0xA0, 0x02, 0x00, 0x02]` — address=`0xA0` → this router, count=2, then bytes `0x00` and `0x02`. This looks correct.

But there's a subtlety: the `init_router` in `ROUTER_TYPE=1` (upper) uses `header_flit[PORT_BITS+LOWER_BITS:LOWER_BITS]` for port_select when routing to lower routers. `PORTS=8, PORT_BITS=$clog2(8)=3, LOWER_BITS=$clog2(4)=2`. So for lower routing: `port_select = header_flit[4:2]`. For the lower router address `0x80` = `1000_0000`: `port_select = 0b000_00[4:2] = 0`. But `0x80 = 1000_0000`, bits `[4:2]` = `000`. That routes to port 0 of the upper router. ✓

The key problem is that the `ready_in` handshake in the init stream driver: when `ready_in` is low (router busy), bytes get dropped. Let me add more diagnostic to see if the SF8 map actually got loaded:

```
Made changes.
```

Summarized conversation historyI'll pick up where we left off - running the testbench with the SF8/SF4 controller monitors to see if the forwarding maps are being written correctly. Now let me run the testbench to see the current output:

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot: Let me run the testbench and see the output: Good, no errors. Now run it: The problem is crystal clear now. **SF8 reads `0x000` from the main FIFO** at @60850 — not `0x7C0`. The spike `0x7C0` was written at @60850 but read as `0x000`. This means the spike was written *after* the main FIFO's read cycle completed, or the spike got lost because `accelerator_done` was already asserted before we injected the spike.

Let me check the timing more carefully - the `accelerator_done` was hit immediately after injecting the spike. Let me look at the neuron_accelerator.v to understand the `accelerator_done` signal and the timing of main FIFO reads: The key insight from the trace: **SF8 reads `0x000` from main_fifo at @60850**, right when `accelerator_done` is asserted. This tells me:

1. The spike `0x7C0` was written to main FIFO at @60850
2. BUT `accelerator_done` was *already* true at that moment (main FIFO empty=1 just before write)
3. The SF8 reads the FIFO in the same cycle, but because `main_fifo_wr_en` and `main_fifo_rd_en` hit simultaneously, the FIFO read gets the old (empty) data

The real problem: **`wait(accelerator_done)` returns TRUE before we inject the spike** - then we inject the spike, and the spike immediately flows through but `accelerator_done` is still asserted. Look at the trace: `accelerator_done` was caught at @60850 right when spike was read (0x000).

Actually the sequence is: `wait(accelerator_done)` resolves → inject spike (@60750 region) → SF8 reads immediately → gets wrong data because FIFO write and read are simultaneous.

Let me check when the spike was actually written vs when the FIFO was read: Now I can see the issue precisely. There's a **huge timing gap** - the SF8 map writes happen around @8000, but then nothing happens until @60850. This means:

1. The init bytes complete around @8000 (SF8/SF4 maps written)
2. Then there are still weight/neuron init bytes to process (the 188 bytes take until @60000+)
3. The SF8 reads from main_fifo at @60850 reading `0x000` - but this is the **empty FIFO signal firing the SF8 "nothing to do" read**, not our spike

The real problem: The spike (`0x7C0`) was written to main FIFO, but **SF8 read the FIFO while it was already in an ongoing read cycle** that returned 0x000. The `accelerator_done` becomes true while the init stream is still being processed by internal modules.

Let me look at what the spike_forwarder_8 does when `accelerator_done` is already asserted: Now I understand the spike_forwarder.v. The key is the forwarding logic. The `eligible[0]` is:

```
eligible[0] = !main_fifo_empty_out && ~|(forwarding_map[0] &
~fifo_space_available);
```

So `forwarding_map[0]` needs to be set correctly. The SF8 maps are:

- `forwarding_map[0]` = `0b000000010` (row 0: main → port 0 = group 0) ✓
- `forwarding_map[1]` = `0b000000001` (row 1: port 0 = group 0 → main) ✓

The trace confirms these were written. But the spike read as `0x000`. Let me look at what happens: **The spike `0x7C0` is in `main_fifo_in`, but SF8 reads from `main_fifo_in.dout`**.

The issue: `network_mode` is toggled back to 0 after init! Look at the init driver:

```
network_mode <= 0; // exit init mode
```

But the spike_forwarder uses `router_mode` (which comes from `network_mode`). **When `router_mode=0`, the forwarding_map stays loaded, but does the forwarder still route?** Looking at spike_forwarder.v:

```
else if (router_mode && load_row) begin
    // load row logic
```

And `eligible[0]` doesn't depend on `router_mode`. So routing should work even in mode 0.

Wait, let me re-read the trace. SF8 reads the main FIFO but gets `0x000`. The FIFO is FWFT (first-word fall-through)? Let me check: **Found the bug!** The FIFO is **NOT** first-word fall-through. It's a **registered read**: `dout` is only valid the cycle **after** `rd_en` is asserted. When `rd_en` is not asserted, `dout <= 0`.

So the timing is:

1. Cycle N: Spike `0x7C0` written to main_fifo_in
2. Cycle N+1: `main_fifo_empty_in` goes low, `eligible[0]` becomes true, `selected_port[0]=1`, `store_selected_port` = 0 (it reads main FIFO: `main_fifo_rd_en_out=1`) → But FIFO needs 1 more cycle for dout
3. Cycle N+2: `main_out_data_out` (= FIFO dout) has `0x7C0`. But `store_selected_port` was stored as 0 in cycle N+1, so in cycle N+2 it correctly reads `main_out_data_out`

Wait - the trace shows SF8 reads `0x000` at @60850. Let me reconsider: **Is the spike actually being written to the FIFO at all?** The testbench does:

```
@(posedge clk); #1;
main_fifo_din_in <= 11'h7C0;
main_fifo_wr_en_in <= 1;
@(posedge clk); #1;
main_fifo_wr_en_in <= 0;
```

This SHOULD write the spike. But the `accelerator_done` becomes true immediately after (because at the moment of `wait(accelerator_done)` resolution, `main_fifo_empty_in=1`). Then we set `main_fifo_wr_en_in=1` in the same time step where `accelerator_done` becomes true...

Actually looking again at the trace: The `accelerator_done` wait resolves (`[PHASE 2] accelerator_done — spike fully processed`) — then we inject the spike — then `SF8 read from main_fifo: data=0x000 @60850` — this is ONE read event that gives 0x000. Then `accelerator_done` is asserted AGAIN.

The real question is: **why does SF8 read the FIFO and get 0x000?** With a registered FIFO:

- Cycle 1: `rd_en=1` on empty FIFO → `do_read=0` → `dout=0`

- OR: `rd_en=1` on non-empty FIFO but spike was just written same cycle → write happens but read sees old empty state

Let me add more precise timing to the trace. I need to add a monitor for `main_fifo_wr_en_in` and `main_fifo_empty_in` to the TB:

Actually, let me approach differently. The core issue is:

1. We write the spike to `main_fifo_in`
2. The SF8 reads `main_fifo_dout_in`
3. But when `do_read` happens with `main_fifo_empty_in=0` (FIFO has spike), the **same cycle** the `main_fifo_rd_en_in=1`, the **next cycle** `dout` becomes valid

BUT — the TB trace shows `SF8 read from main_fifo: data=0x000` — this trace fires when `main_fifo_rd_en_in && !main_fifo_empty_in`. If FIFO is not empty but dout=0... that would mean the trace fires when the FIFO is empty (`!empty` is false, meaning `do_read=0`, but `rd_en=1`).

Wait, let me re-read the trace code:

```
if (dut.main_fifo_rd_en_in && !dut.main_fifo_empty_in)
    $display("[TRACE] SF8 read from main_fifo: data=0x%03h",
dut.main_fifo_dout_in);
```

This fires when `rd_en && !empty`, which means data was actually read. The data shown is `main_fifo_dout_in` which is `dout` of the FIFO — but with this registered FIFO, `dout` hasn't been updated yet (it'll be valid NEXT cycle)! The trace is showing the **current** (old) dout, which is 0.

So the spike IS being read, but the trace monitor captures the wrong cycle. However, the actual forwarding:

```
// Pipeline stage 2: Forward data
if (store_selected_port > -1) begin
    ...
    main_in_data_in <= forwarding_map[0][0] ? main_out_data_out : 0;
```

`main_out_data_out` = `main_fifo_dout_in`. This is captured one cycle AFTER `rd_en`. So:

- Cycle N: `selected_port[0]=1`, `rd_en=1`, `store_selected_port` gets set to 0
- Cycle N+1: `main_out_data_out` = dout = `0x7C0` (valid now), `store_selected_port=0` → writes to SF4 FIFO

But the trace at stage 2 shows `SF4[0] reads from SF8 FIFO: data=0x000`!

This means the SF4 FIFO was read empty. Let me check if `forwarder_8_fifo_wr_en_in[0]` (SF8→SF4) ever fires with real data: None of stages 4-9 triggered at all — the SF8→SF4 write never happened. So the spike packet **never got forwarded** from SF8 to SF4.

Let me look at the `accelerator_done` timing more carefully. The problem might be that `accelerator_done` goes true AFTER reset but BEFORE init completes, and the testbench moves on to

inject the spike before any routing happens. Let me add more trace to actually see what's happening with the main_fifo write/empty signal: I see the problem now. The Stage 1 trace fires at @60850 but shows `data=0x000`. This is because the FIFO dout is still 0 at the time of the read (registered output takes one extra cycle). However, the pipeline stage 2 in spike_forwarder.v should STILL correctly forward it next cycle using `main_out_data_out`.

But Stage 2 (SF8→SF4 write) never fires. This means either:

1. `forwarding_map[0]` is not set (but SF8 map written row=0 fwd_row=0b000000010 ✓)
2. The eligible/selected logic doesn't work
3. `network_mode` was set back to 0 BEFORE the maps were loaded (the maps require `router_mode=1`)

Wait — look at the spike_forwarder code:

```
else if (router_mode && load_row) begin
    forwarding_map[row_index] <= forwarding_row;
```

The maps ARE loaded (`router_mode=1`) during init. They persist. Good.

But wait — the maps were written at @2550 and @7850 (during init). After init, `network_mode` was set to 0. The `router_mode` input to spike_forwarder = `network_mode`. When `router_mode=0` and `load_row=0`, the else branch runs which just does normal forwarding. So routing still works.

Let me look more carefully at what's different. **The SF4 group 0 forwarding_map row 0**: `fwd_row=0b00010`. This means bit 1 is set — which maps to port 0 (cluster 0). But in spike_forwarder_4, `forwarding_map[0]` maps main→port 0.

The crucial question: What is the `row_index` space for the SF8 forwarding map?

The spike packet is `0x7C0 = 0b11111000000` = {cluster_id=62, neuron_id=0}.

For SF8, the row is selected based on the spike SOURCE. The SF8 looks at some bits of the packet to choose which forwarding_map row. Let me look at the SF8 controller: Now let me look at the spike_forwarder.v to understand HOW the forwarding_map is indexed during operation (not init). The spike packet routes through `forwarding_map[port_index]` — but what determines WHICH row is used? Now I see the **critical bug**. The spike_forwarder.v uses `forwarding_map[port_index]` where `port_index = store_selected_port` — and `store_selected_port` = the **port that is currently eligible** (port 0 = main FIFO). So `forwarding_map[0]` maps the main FIFO to its destination.

`forwarding_map[0] = 0b000000010` means bit 1 is set → goes to `fifo_wr_en_out[0]` (SF4 group 0 FIFO).

This is correct. But the problem is: **When does the SF8 see `eligible[0]=1`?**

`eligible[0] = !main_fifo_empty_out && ~|(forwarding_map[0] & ~fifo_space_available)`

The SF8's `main_fifo_empty_out` is `dut.main_fifo_empty_in` (the input main FIFO). When spike `0x7C0` is written, `empty` goes from 1→0 **next cycle** (registered FIFO count update). Then:

- Cycle write: `count` updates, `empty=0` next cycle
- Next cycle: `eligible[0]=1`, `selected_port[0]=1`, `main_fifo_rd_en_out=1`
- But `main_fifo_dout_in` is still 0 at this point!

Wait — look at the FIFO again. When `rd_en=1` and `!empty`, in the SAME cycle:

- `dout <= mem[rd_ptr]` — takes effect NEXT clock
- But `count <= count - 1` also NEXT clock

So the sequence with registered FIFO:

- T0: Write spike (`wr_en=1`, `din=0x7C0`), `mem[0]=0x7C0`, `count=1` at T0+1
- T1: `empty=0`, `eligible[0]=1`, `rd_en=1`, `store_selected_port=0`
- T2: `dout=0x7C0` (mem[0] output now valid), `count=0`, `empty=1`
  - But `store_selected_port` was set to 0 at T1, so in T2: `main_in_data_in <= forwarding_map[0][0] ? main_out_data_out : 0` → `main_out_data_out=dout=0x7C0`... But `forwarding_map[0][0]` = bit 0 = 0 (only bit 1 is set: 0b10)!

That's the bug. `forwarding_map[0] = 0b000000010`. When forwarding from port 0 (main) to port L (SF4 group 0):

- `fifo_wr_en_out[l] <= forwarding_map[0][l+1]` → for l=0, bit 1 = 1 ✓

So it DOES write to SF4 FIFO. Let me re-read the forward code:

```
if (store_selected_port > -1) begin
    // store_selected_port = 0 (came from main FIFO)
    if (store_selected_port > 0) begin  // FALSE (0 > 0 is false)
        ...
    end else begin   // THIS BRANCH
        main_in_data_in <= forwarding_map[0][0] ? main_out_data_out : 0;
    end
    main_fifo_wr_en_in <= forwarding_map[0][0];  // bit 0 = 0, so NO write
back to main

    for (l=0; l<num_ports; l=l+1) begin
        if (store_selected_port > 0) begin // FALSE
            ...
        end else begin  // THIS BRANCH
            fifo_in_data_out[l*data_width +: data_width] <=
forwarding_map[0][l+1] ? main_out_data_out : 0;
        end
        fifo_wr_en_out[l] <= forwarding_map[0][l+1]; // l=0: bit1=1 →
WRITES to SF4[0]!
    end
```

So it SHOULD write `main_out_data_out` = `0x7C0` to `fifo_in_data_out[0]` and assert `fifo_wr_en_out[0]`.

BUT: `main_out_data_out` — what is this? In spike_forwarder_8, it's connected as:

```
.main_out_data_out(main_fifo_dout_in),
```

And `main_fifo_dout_in` is the `dout` of `main_fifo_in`. With registered FIFO, at T2 when `store_selected_port=0`:

- `main_fifo_dout_in` = 0x7C0 ✓ (available now because T1 had rd_en=1)

So it SHOULD work! The trace shows stage 2 (SF8→SF4 write) never fired though. Let me check if there's an intermediate issue. Maybe the `eligible[0]` evaluates differently.

Let me add a targeted trace to check `eligible[0]`, `selected_port`, and `store_selected_port` right when the spike is in the FIFO:

```
Made changes.
```

Excellent! Now I can see clearly:

1. **@60750**: Spike `0x7C0` written to main_fifo. `main_fifo_empty_in` goes 0.
2. **@60850**: `eligible[0]=1`, `selected[0]=1`, `fwdmap[0]=000000010`, `store_sel=-1`, `rd_en=1`.
   - `main_fifo_empty_in` goes back to 1 (FIFO was read)
   - `dout=0x000` (registered FIFO — dout is the OLD value, actual data available NEXT cycle)
3. **@60950**: This should be where `store_selected_port=0` and `main_out_data_out=0x7C0` — but the trace shows NO SF8→SF4 write!

The `store_sel=-1` at @60850 is the KEY issue. At T=60850:

- `selected_port[0]=1` → `current_selected_port_index = 0`
- But `store_selected_port` (registered, set last cycle) = **-1** (from previous idle cycle)

So at T=60850: `store_selected_port` is still -1 → the **forwarding** (`fifo_wr_en_out[l]`) uses the **previous** `store_selected_port=-1` → no write to SF4.

At T=60950:

- `store_selected_port` should be 0 (updated from T=60850)
- `main_out_data_out = dout = 0x7C0` (valid now)
- `fifo_wr_en_out[0] = forwarding_map[0][1] = 1` → **SHOULD write to SF4!**

But we saw that at @60850, `main_fifo_empty_in` changed to 1 — the FIFO is empty again. At @60950, is the forwarding code still executed?

Looking at spike_forwarder.v:

```
else begin // (not rst, not router_mode+load_row)
    // Update registered inflight_count
    for (l=0; l<num_ports+1; l=l+1)
        inflight_count_reg[l] <= inflight_count[l];

    // Pipeline stage 1: Store selected port
    store_selected_port <= current_selected_port_index;

    // Pipeline stage 2: Forward data
    if (store_selected_port > -1) begin
```

At T=60950:

- `store_selected_port=0` (updated from T=60850) → enters `if (store_selected_port > -1)` ✓
- `main_out_data_out = 0x7C0` ✓
- `forwarding_map[0][1]=1` → `fifo_wr_en_out[0]=1` → **SF4[0] FIFO write**

But the trace shows NO `SF8→SF4[0] write` at all! So something prevents it. The problem must be that at T=60950, `accelerator_done` is already true and we detect it, then `time_step` is asserted.

Wait — looking at the timing more carefully: at @60850 the `accelerator_done` fires AND `main_fifo_empty_in` goes to 1. The `wait(accelerator_done)` is a combinational check - it fires immediately when the signal goes high.

The real issue: `accelerator_done` includes `main_fifo_empty_in`. As soon as the FIFO read empties the FIFO at @60850, `accelerator_done` goes high. But the forwarding (writing to SF4) only happens at @60950 — one cycle LATER.

So the TB detects `accelerator_done` at @60850 and immediately asserts `time_step` at @61250 (after some `@(posedge clk); #1` delays). Meanwhile at @60950, the SF8 SHOULD write `0x7C0` to SF4...

But wait: the `wait(accelerator_done)` in the TB is followed by `repeat(3) @(posedge clk);` before asserting `time_step`. So:

- @60850: `accelerator_done` fires, `wait()` resolves
- @60850..@61250: 3+ more cycles of `@(posedge clk)` pass
- @61250: `time_step` asserted

That's enough time for @60950 to happen. But the spike is GONE already (empty FIFO). Actually, wait — the `inflight_count` mechanism should handle this. Let me check what `inflight_count` does.

At @60850: `selected_port[0]=1` and `forwarding_map[0][1]=1` → `inflight_count[1]` should be incremented! This means the SF8 forwarding considers a packet "in flight" to port 1. The `done` signal:

```
assign done = &fifo_empty_in && &fifo_empty_out && zero_inflight;
```

So `forwarder_8_done` should be 0 until the inflight packet is resolved, and `accelerator_done` should NOT be true at @60850!

Let me check: `accelerator_done = all_clusters_done & all_forwarders_done & main_fifo_empty_in & main_fifo_empty_out & resolvers_done`

`all_forwarders_done = &(forwarder_4_done | done_mask) && forwarder_8_done`

If `forwarder_8_done=0` at @60850 (due to inflight_count[1]=1), then `accelerator_done` should NOT be true!

But the trace shows `[PHASE 2] accelerator_done` triggers at @60850. Let me check — is `inflight_count[1]` actually being incremented?

Looking at the inflight logic:

```
for (l=0; l<num_ports+1; l=l+1) begin
    if (|selected_port && forwarding_map[current_selected_port_index][l] &&
        forwarded_port > -1 && forwarding_map[forwarded_port][l]) begin
        // cancel
    end else if (|selected_port &&
forwarding_map[current_selected_port_index][l]) begin
        inflight_count[l] <= inflight_count[l] + 1;  // increment
    end else if ...
end
```

At T=60850:

- `|selected_port = 1`
- `current_selected_port_index = 0`
- `forwarding_map[0][l]`: for l=1, bit 1 = 1 → condition is true
- `forwarded_port = -1` (still -1 from last idle)
- So: `inflight_count[1] <= inflight_count[1] + 1 = 1`

This increment happens at @60850+clock edge = @60950. So at @60850, `inflight_count[1]` is still 0, `zero_inflight=1`, `done=1`. That's why `accelerator_done` fires!

The inflight counter lags by one clock! At the same cycle the read fires, `zero_inflight=1` (since inflight hasn't been updated yet). This is a **design flaw** in the spike_forwarder — `done` is asserted prematurely while data is in transit.

However — looking at the neuron_cluster_tb test, this works fine. So perhaps the problem is that in the cluster test, there's no need for forwarding (the packet goes directly to the cluster). Let me think about this differently.

The real fix needed: **Don't rely on `accelerator_done` to determine when the spike has been fully processed.** Instead, wait for the spike to actually reach the cluster. Or better: just wait longer after injecting the spike.

But there's a deeper issue: even if `time_step` is asserted after the SF8→SF4 write (@60950), the SF4 pipeline still needs ~4 more cycles to resolve (SF4 read → cluster FIFO write → incoming_forwarder match → weight resolver). So `time_step` at @61250 is still too early.

The real fix is: **The testbench should wait for a proper "all done" signal that includes all inflight packets**. But the current design has this race condition.

Let me look at what the real neuron_accelerator_tb does - it uses the same `accelerator_done`: I found the key difference! In neuron_accelerator_tb.v:

```
if (dump_done) begin
    ...
    time_step <= 1;
end
```

The real TB waits for `dump_done` before asserting `time_step`. In our known_value_dump_tb.v we wait for `accelerator_done` first, which fires too early (before the spike has been fully processed through the pipeline).

The real TB feeds spikes clock-by-clock from an `always @(posedge clk)` block, then waits for `dump_done` before proceeding. This means the spike had time to go through the entire pipeline.

The fundamental issue is: **`accelerator_done` fires the moment the main FIFO becomes empty — even before the spike has been forwarded to SF4 → cluster → weight_resolver → accumulator**. The `inflight_count` mechanism in spike_forwarder.v is supposed to prevent this but there's a 1-cycle lag.

Actually wait, looking more carefully at the trace:

- @60750: spike written, FIFO count goes to 1
- @60850: SF8 reads FIFO (rd_en=1), FIFO count goes to 0, empty→1
- The inflight increment ALSO happens at @60850 posedge
- At @60850, `done = &fifo_empty_in && &fifo_empty_out && zero_inflight` - but these are combinational from `inflight_count`, which was NOT yet updated (the update is registered, takes effect at @60950)

So the issue is that `done` (and thus `forwarder_8_done`) uses `inflight_count` which updates at the same clock edge when the read fires, so for ONE cycle `done` sees zero inflight even though there IS inflight data. `accelerator_done` pulses true for exactly that one cycle.

The fix: **In our testbench, instead of `wait(accelerator_done)`, we should either**:

1. Wait for `dump_done` (like the real TB does)
2. Wait for `accelerator_done` twice (it dips, then rises again after inflight resolves)

Actually the real TB pattern is even simpler: it writes all spikes FIRST (in the same `always @(posedge clk)` block), then **waits for `dump_done`** of the PREVIOUS timestep before asserting `time_step`. So spikes are always fully processed before time_step.

For our TB: we inject the spike, then we should wait LONGER — not for `accelerator_done` (which is glitchy) but either wait for a fixed number of cycles or watch for `dbg_all_clusters_done`.

Let me look at what `dbg_all_clusters_done` means vs `accelerator_done`:

```
assign all_clusters_done = &(cluster_4_done | done_mask);
assign accelerator_done = all_clusters_done & all_forwarders_done &
main_fifo_empty_in & main_fifo_empty_out & resolvers_done;
```

`all_clusters_done` = all neuron computation done. This is what we want after `time_step`.

The proper TB flow should be:

1. Inject spike
2. Wait for spike to be consumed (all FIFOs empty AND no inflight) — this is `accelerator_done` but with the glitch
3. Assert `time_step`
4. Wait for `all_clusters_done` (neuron computation finished)
5. Wait for `dump_done` (dump FSM finished writing v_pre_spike)

The glitch in `accelerator_done` at @60850 (1 cycle pulse) means our `wait(accelerator_done)` catches that first glitch. The real data is processed at @60950 onward.

The simplest fix: After injecting the spike, add a `repeat(20)` delay to give the pipeline time to fully process the spike before checking `accelerator_done`. Or, better, wait for `accelerator_done` to be stable high for 2 cycles.

Actually the best fix: mimic the real TB - feed spikes in a registered `always @(posedge clk)` block, then wait for `dump_done` before `time_step`. Let me restructure the Phase 2 logic: The key change needed is: **after injecting the spike, wait enough cycles for the spike to fully propagate through the entire pipeline BEFORE the `accelerator_done` glitch fires**. The problem is `wait(accelerator_done)` catches the 1-cycle glitch at @60850.

The best approach: **Wait for `accelerator_done` to be stable** — i.e., wait for it to go low then go high again, OR simply use `repeat(50)` after spike injection.

But more importantly, the real TB doesn't check `accelerator_done` at all for the "pre-time_step" wait — it just feeds spikes clock-by-clock then waits for `dump_done`. Let me restructure Phase 2 to:

1. Inject spike
2. Wait a generous number of cycles (50) for pipeline to drain
3. Check that `accelerator_done` is stable high
4. Assert `time_step`
5. Wait for `dump_done` (which implies neuron computation AND dump FSM completed)

```
Made changes.
```

Good progress — the timing fix worked. But Stage 2 (SF8→SF4 write) still never shows up. The trace shows:

- SF8 reads main_fifo at @60850 (rd_en=1, dout=0x000 still old value)
- SF4[0] reads from SF8 FIFO at @61150 — but data=0x000!
- Cluster0 incoming_fwd rd_en at @61450 — also 0x000!

The pipeline IS flowing (all 3 stages fire), but all data is 0x000. This confirms the spike data is being LOST. Let me understand why `dout=0x000` at @61150 when SF4 reads from SF8 FIFO.

The SF8→SF4 write (stage 2) never appeared in the trace, which means `forwarder_8_fifo_wr_en_in[0]` never fired. But SF4 still reads from that FIFO... and gets 0x000. That's the FIFO reading 0 when empty (registered FIFO).

Wait — actually looking at the trace timeline:

- @60850: SF8 reads main_fifo, `main_fifo_empty_in → 1`
- @61150: SF4 reads SF8 FIFO — this is 3 cycles after @60850. But where's the SF8→SF4 write?

The SF8→SF4 write should happen at @60950 (1 cycle after @60850). But it never appeared. Why?

Looking at spike_forwarder.v again — at @60850:

- `store_selected_port` is currently -1 (from previous idle cycle)
- In this cycle: `store_selected_port <= current_selected_port_index = 0` (registered)

At @60950:

- `store_selected_port = 0` → enters `if (store_selected_port > -1)` branch
- `main_out_data_out = dout = 0x7C0` (valid now from FIFO)
- `fifo_wr_en_out[0] = forwarding_map[0][1] = 1` → WRITES to SF4 FIFO at @60950!

But the trace shows `SF8→SF4[0] write` never fires. This means `forwarder_8_fifo_wr_en_in[0]` is never 1. Wait — let me check if the SF8 forwarding actually goes to `fifo_wr_en_out[0]` in spike_forwarder.v, and if that maps to `forwarder_8_fifo_wr_en_in[0]` in neuron_accelerator.v: I see the SF8 `fifo_wr_en_out` wiring: `.fifo_wr_en_in(forwarder_8_fifo_wr_en_in)` - so `forwarder_8_fifo_wr_en_in[0]` is the signal from SF8 to SF4 group 0.

But wait — `fifo_wr_en_in` in spike_forwarder_8 maps to `fifo_wr_en_in` in spike_forwarder.v, which corresponds to the FIFOs that RECEIVE data from OUTSIDE (the "in" FIFOs).

Let me re-read the spike_forwarder ports:

```
input wire [num_ports-1:0] fifo_wr_en_in,      // from outside → in-FIFOs
output wire [num_ports-1:0] fifo_rd_en_out,    // read out-FIFOs (to
outside)
input wire [num_ports*data_width-1:0] fifo_in_data_in,   // data from
outside
```

```
output wire [num_ports*data_width-1:0] fifo_out_data_out, // data to
outside
```

And the internal FIFOs:

```
fifo_gen_in: wr_en = fifo_wr_en_in, rd_en = fifo_rd_en_in (internal)
fifo_gen_out: wr_en = fifo_wr_en_out (internal), rd_en = fifo_rd_en_out
```

The forwarding outputs go via `fifo_wr_en_out[l]` to the **out** FIFOs. These are read by external `fifo_rd_en_out`.

In `spike_forwarder_8`, the port 0 (`fifo_wr_en_out[0]`) writes to the out-FIFO[0], which is read by `fifo_rd_en_out[0]` = `forwarder_8_fifo_rd_en_out[0]`. But `forwarder_8_fifo_wr_en_in` is the signal going INTO spike_forwarder_8 (from SF4 group i → SF8 main FIFO direction).

So the TB trace is monitoring `forwarder_8_fifo_wr_en_in[0]` — but that's the WRONG direction! It monitors data going FROM SF4 group 0 INTO SF8, not FROM SF8 INTO SF4.

The correct signal to monitor for SF8→SF4 is `forwarder_8_fifo_rd_en_out[0]`! Or better, monitor the out-FIFOs of spike_forwarder_8 directly.

Let me fix the trace to monitor the right signal:

Actually, looking at spike_forwarder_8.v, the port connections:

```
.fifo_rd_en_out(forwarder_8_fifo_rd_en_out),   // spike_forwarder reads
out-FIFOs
.fifo_wr_en_in(forwarder_8_fifo_wr_en_in),     // spike_forwarder writes
in-FIFOs
.fifo_in_data_in(forwarder_8_fifo_in_data_in), // data into in-FIFOs (from
SF4)
.fifo_out_data_out(forwarder_8_fifo_out_data_out), // data from out-FIFOs
(to SF4)
```

In neuron_accelerator.v, these SF8 FIFOs are connected to SF4 like this:

- SF8's **out-FIFO[i]** → read by SF4[i]'s spike forwarder main FIFO: `forwarder_8_fifo_out_data_out[i]`
- SF4[i] reads them via `forwarder_8_fifo_rd_en_out[i]`

So Stage 3 trace (`forwarder_8_fifo_rd_en_out[0]`) is correct — SF4 reads from SF8 out-FIFO.

But Stage 2 trace (`forwarder_8_fifo_wr_en_in[0]`) is monitoring SF4→SF8 direction (wrong). The SF8→SF4 write uses `fifo_wr_en_out` which is **internal** to spike_forwarder.v.

The actual question is: did spike_forwarder WRITE to its out-FIFO[0]? I need to monitor:

```
dut.spike_forwarder.spike_forwarder_inst.fifo_wr_en_out[0]
```

Let me update the Stage 2 trace:

```
Made changes.
```

Also fix Stage 4 — the SF4 writes to cluster: let me check the correct internal signal. The SF4[0] group's spike_forwarder_4 is `dut.gen_spike_forwarder_4[0].spike_forwarder_4_inst.spike_forwarder_inst`. Let me also monitor `main_out_data_out` at the time of SF8 write to confirm data:

```
Made changes.
```

Now we can see the full pipeline:

1. @60750: spike `0x7C0` written to main_fifo ✓
2. @60850: SF8 reads main_fifo (dout still 0x000 - old registered value)
3. **@61050: SF8 writes `0x7C0` to out-FIFO[0]** ✓ — data IS correct!
4. @61150: SF4[0] reads from SF8 FIFO — but gets `0x000` (registered FIFO, dout not valid yet)
5. **@61350: SF4[0] writes `0x7C0` to out-FIFO[0]** ✓ — data IS correct!
6. @61450: Cluster0 reads from SF4 FIFO — but packet `0x000`!

The same pattern repeats at each stage: the **read** trace shows 0x000 because that's the registered FIFO dout at the moment of rd_en (not yet updated), but the **write** shows `0x7C0` correctly one cycle later.

So the pipeline IS working correctly, and the data `0x7C0` does reach Cluster0. The question is what happens at Stage 5+ (weight resolver, accumulator, etc.) and whether neuron actually fires.

The problem is further downstream — the `time_step` is asserted at @66050 (50 cycles after spike injection), and by then the weight should have been loaded. Let me add traces for the weight resolver and cluster accumulation stages. Also, let me check if the incoming_forwarder matches the packet correctly.

Let me look at the incoming_forwarder:

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot: Let me continue analyzing. The pipeline trace shows data `0x7C0` reaches cluster0, but the cluster reads `0x000` (same registered FIFO issue). Let me check the weight resolver and accumulator path more carefully. Now let me look at the `weight_memory.v` to understand the init address format: `weight_memory` has an **asynchronous read** (`assign weight_out = weight_mem[weight_addr]`) — so there's no clock latency for reads. Good.

Now let me understand the `weight_resolver` address scheme. Looking at incoming_forwarder.v:

```
weight_addr_out <= base_weight_addr_reg + (found_row_index << 5) +
packet[4:0];
```

And the init protocol:

- `OPCODE_LOAD_IF_BASE = 0x02`: sets `base_weight_addr_reg`
- `OPCODE_LOAD_IF_ADDR = 0x03`: stores cluster_id into `cluster_id_table[row_index]`

The spike packet `0x7C0 = {cluster_id=62, neuron_id=0}`. The incoming_forwarder matches `packet[10:5] = 6'd62` against `cluster_id_table`. If `cluster_id_table[0] = 62`, then `found_row_index=0`, `weight_addr = base + 0*32 + 0 = base = 0`.

Then the weight_resolver gets `buffer_addr_in = 0`, reads `weight_mem[0]`, and outputs the entire 32-neuron weight row. Weight `[0]=5.0`, `[1]=2.0`.

Key question: Did `cluster_id_table[0]` get set to 62? Let me check the init bytes:

PKT 6: `init_rom[i]=0x00; count=2; 0x03 (LOAD_IF_ADDR); 0x3E (cluster 62)`.

But wait — `cluster_id_table[i]` is initialized to `6'h3F` (63, not 62). The table entry is compared to `packet[10:5]`. The packet `0x7C0 = 11'b11111000000`, so `packet[10:5] = 6'b111110 = 62`. And `cluster_id_table[i]` check: `cluster_id_table[i] != 6'h3F && cluster_id_table[i] == packet[10:5]`. So `cluster_id_table[0]` needs to be `6'd62 = 0x3E`.

Looking at the init byte for LOAD_IF_ADDR: the cluster byte in PKT 6 is `0x3E`. In incoming_forwarder.v:

```
cluster_id_table[row_index] <= {1'b0, cluster_id[5:0]};
```

`cluster_id` comes from the cluster_controller parsing. Let me check the cluster_controller: Good. The cluster_controller parses init bytes. Now I need to understand the `self_data_mng.v` to see how bytes are routed to the cluster_controller vs weight_resolver: Now I see a critical issue! `self_data_mng` state machine:

- **State 0**: reads `port_selected = data_in[0]` (bit 0, NOT the full byte value)
- **State 1**: reads `count`
- **State 2**: forwards `count` bytes to the selected port

In our PKT 3 (SF4 row 0 init):

```
init_rom[i]=8'h80; // router addr = lower router 0 self
init_rom[i]=8'h03; // count = 3 bytes follow
init_rom[i]=8'h00; // sdm: port_select=0 → bit 0 = 0 → forwarder
init_rom[i]=8'h01; // sdm: count=1
init_rom[i]=8'h02; // sf4 byte
```

So `self_data_mng` receives `[0x00, 0x01, 0x02]`:

- State 0: `port_selected = 0x00[0] = 0` → forwarder ✓
- State 1: `count = 0x01 = 1` ✓
- State 2: forwards `0x02` to forwarder ✓

And PKT 9 (weight resolver):

```
init_rom[i]=8'h80; // addr
init_rom[i]=8'h85; // 133 bytes follow
init_rom[i]=8'h01; // sdm: port_select → bit 0 = 1 → resolver ✓
init_rom[i]=8'h83; // sdm: count = 131 ✓
// then 131 weight_resolver bytes
```

This looks correct. But wait — the `init_router.v` routes to the lower router self (address `0x80`). How does the `init_router` deliver bytes to `self_data_mng`? Let me check: Now I have enough understanding. Let me look at the neuron_accelerator.v to understand how the lower init_router connects to self_data_mng and cluster: Now I have the full picture. Let me check the `weight_resolver` init protocol more carefully. Looking at weight_resolver.v state machine:

- **State 0**: reads `weight_addr_init[7:0]` (addr_low)
- **State 1**: reads `weight_addr_init[15:8]` (addr_high)
- **State 2**: reads `weight_flit_count`
- **State 3**: reads weight data bytes until `flit_count-1` flits received, then `load_weight_mem=1`

The `weight_flit_count` here is the **total number of bytes** for the weights (not neurons). For 32 neurons × 4 bytes = 128 = `0x80`.

In our PKT 9 init bytes:

```
[sdm payload] addr_low=0x00, addr_high=0x00, flit_count=0x80, then 128
bytes of weights
```

The `weight_resolver` receives these bytes in order:

- `data=0x00, load_data=1` → state 0 → `weight_addr_init[7:0] = 0x00`
- `data=0x00, load_data=1` → state 1 → `weight_addr_init[15:8] = 0x00`
- `data=0x80, load_data=1` → state 2 → `weight_flit_count = 0x80 = 128`
- `data=0x00, 0x00, 0x05, 0x00` (N0: 5.0 LE) → state 3...
- `data=0x00, 0x00, 0x02, 0x00` (N1: 2.0 LE) → state 3...

- ... 120 more zero bytes ...

At counter=127 (== flit_count-1=127): `load_weight_mem=1`, `weight_mem[0]` gets the full 128-byte weight row.

The weight is stored as `weight_in_mem[8*weight_flit_counter +: 8] = data`. So weight_in_mem builds up byte by byte:

- `weight_in_mem[7:0] = 0x00` (N0 byte 0)
- `weight_in_mem[15:8] = 0x00` (N0 byte 1)
- `weight_in_mem[23:16] = 0x05` (N0 byte 2 = 0x05)
- `weight_in_mem[31:24] = 0x00` (N0 byte 3)
- → `weight_in_mem[31:0] = 0x00050000` ✓

This should work. But there's something subtle: the weight_resolver's `chip_mode` input:

```
.chip_mode(network_mode),
```

And in weight_resolver:

```
end else if(chip_mode && load_data) begin
    case (state) ...
end else begin
    store_selected_buffer <= selected_buffer;
    ...
end
```

So `chip_mode=1` (network_mode=1) is needed to init weights. Our init stream sets `network_mode=1` before streaming, so this is correct.

The `weight_addr_in_mem` assignment:

```
assign weight_addr_in_mem = chip_mode ? weight_addr_init :
|store_selected_buffer ? buffer_addr_out[...] : 0;
```

During init (chip_mode=1): uses `weight_addr_init`. After init (chip_mode=0): uses `buffer_addr_out`.

So weight memory read during inference uses the buffer address. The incoming_forwarder pushes address 0 into buffer[0], and weight_resolver pops it and reads `weight_mem[0]`.

Let me now add targeted traces for weight_resolver init and the cluster's accumulator. The key diagnostic is: did `weight_mem[0]` actually get written with the correct data?

Let me add more specific trace monitors to check the weight_resolver init bytes and the incoming_forwarder matching: Now let me replace all those stages with much better, focused diagnostics that check the init path AND the inference path together:

```
Made changes.
```

Also remove the dump display block that prints every write (it's too verbose), and replace with just non-zero writes: Good. Now remove the verbose DUMP display block that's in the `portb_we && portb_en` always block (lines ~250 previously). Let me find and remove it:

```
Made changes.
```

The `incoming_forwarder` signals aren't directly accessible at this hierarchy — they're likely inside `neuron_cluster.v`. Let me check: The `incoming_forwarder` instance in neuron_cluster.v is named `incoming_forwarder` (same as module name). Let me fix the hierarchy path and also note that `incoming_forwarder` uses `internal_clk` = `chip_mode ? clk : clk & cluster_en`. This is a gated clock! During inference (`chip_mode=0`), `internal_clk = clk & cluster_en`. The `cluster_en` is set to 1 in `cluster_controller` after `work_mode` is written. Let me check if `cluster_en` is properly initialized.

Looking at cluster_controller.v:

```
cluster_en <= 0;  // on reset
...
default: begin
    cluster_en <= 1;
    work_mode <= data;
```

So `cluster_en` only goes 1 when an opcode byte is received. During inference with no init bytes going to cluster, `cluster_en` stays at whatever it was last set to during init.

Actually wait — the `cluster_controller` receives init bytes during init mode. The PKT 7/8 (neuron init) goes through:

- `OPCODE_LOAD_NI=0x01` → sets `work_mode=0x01`
- state transitions handle neuron data

The `cluster_en <= 1` happens in the `default` case of `case (work_mode)`, which is when `work_mode=0`. So when the first byte of a new packet arrives and `work_mode=0`, it sets `cluster_en=1`. That means after the first init opcode byte is received, `cluster_en=1` forever.

But then `internal_clk = clk & cluster_en = clk & 1 = clk`. So the gated clock issue is resolved.

Now let me fix the hierarchy path for `incoming_forwarder`. The instance is named `incoming_forwarder` in neuron_cluster.v:

```
dut.gen_spike_forwarder_4[0].gen_neuron_cluster[0].neuron_cluster_inst.incoming_forwarder
```

(not `.incoming_forwarder_inst`!)

```
Made changes.
```

```
Made changes.
```

Excellent! Now I can see two critical bugs:

1. **`[INIT] Cluster0 IF load_cluster_index: cl_id=2`** — The cluster_id stored is **2**, not **62**! Our init byte for PKT 6 sends `0x3E` (62), but the controller received `cl_id=2`.
2. **Weight resolver init never shows** — No `[INIT] WR[0] mem[...]` line! The weights were never written to the weight memory.
3. **No `[INF] WR[0] addr pushed`** — No address pushed to weight resolver buffer.
4. **No `[INF] Cluster0 IF addr_buf_wr`** — The incoming_forwarder never matched.

Let me fix issue 1 first. `cl_id=2` instead of 62. Let me trace the init byte path:

PKT 6: `[0x00, 0x02, 0x03, 0x3E]` → lower router 0 (addr=0x00), count=2, bytes=[0x03, 0x3E]

The lower init_router (port 0 = cluster 0) receives `0x03` then `0x3E`. The `cluster_controller` receives these bytes. With `work_mode=0`, first byte `0x03` goes to `default` case → sets `work_mode=0x03 (OPCODE_LOAD_IF_ADDR)`, `state=1`. Next byte `0x3E=62` → `OPCODE_LOAD_IF_ADDR` case, `state=1` → `if_cluster_id <= data = 0x3E = 62`, `if_load_cluster_index <= 1`.

But the trace shows `cl_id=2`. This means the cluster_controller received a DIFFERENT byte value. Maybe the packet routing is wrong.

Wait — the lower router `init_router_lower` has `ROUTER_TYPE=0`. For type-0, port selection: `port_select <= header_flit[PORT_BITS-1:0]`. With `PORTS=4`, `PORT_BITS=2`. So address `0x00`: `port_select = 0x00[1:0] = 2'b00 = 0`. That's cluster 0. ✓

But address `0x00` might not equal the `ROUTER_ID` of the lower router. For group 0: `ROUTER_ID = (0 * 4) + 0x80 = 0x80`. The lower router checks `if (header_flit == ROUTER_ID)` for `this_router`. `0x00 != 0x80`, so `this_router=0`. So it routes to `port_select = 0x00[1:0] = 0`. ✓

Hmm, but then why is `cl_id=2`? Let me check: maybe the `0x3E` byte is arriving at the cluster but the `cluster_controller` is mis-parsing it because `work_mode` is already set from a previous packet (PKT 5 sets it to `OPCODE_LOAD_IF_BASE=0x02`!).

Look at PKT 5:

```
[0x00, 0x03, 0x02, 0x00, 0x00]
```

Cluster_controller gets: `[0x02, 0x00, 0x00]`

- First byte `0x02` (`work_mode=0`): default → `work_mode=0x02 (LOAD_IF_BASE)`, state=1
- Second byte `0x00`: `LOAD_IF_BASE` case state=1 → `if_base_weight_addr_init[7:0]=0x00`, state=2
- Third byte `0x00`: state=2 → `if_base_weight_addr_init[15:8]=0x00`, `if_load_addr_in=1`, `work_mode=0`, state=0 ✓

Then PKT 6: `[0x03, 0x3E]` → cluster_controller gets `[0x03, 0x3E]`:

- `0x03`: `work_mode=0`, default → `work_mode=0x03`, state=1 ✓
- `0x3E`: `work_mode=0x03`, state=1 → `if_cluster_id <= 0x3E=62` ✓

So the cluster_controller SHOULD receive `cl_id=62`. But `cl_id=2`…

Let me check: the trace shows `cl_id=2` with `@11050`. Let me check what byte `0x3E` looks like at `if_cluster_id`. The `if_cluster_id` wire is `[$clog2(number_of_clusters)-1:0]`. With `number_of_clusters=4` (our testbench): `$clog2(4)=2` bits! So `if_cluster_id` is only **2 bits wide**! `0x3E = 6'b111110`, truncated to 2 bits = `2'b10 = 2`. That's why!

This is the root bug. In the testbench we set `number_of_clusters=4`, but the spike packet uses cluster_id=62 which needs 6 bits.

The cluster_id in the packet is `packet[10:5]` = 6 bits always. The `cluster_id_table` stores `cluster_id_table[i] <= {1'b0, cluster_id[5:0]}` = 7 bits. But `cluster_id` port in cluster_controller is `[$clog2(number_of_clusters)-1:0]` = only 2 bits wide with N_CLUSTERS=4.

The fix: the packet uses cluster 62 which doesn't fit in 2-bit cluster ID space. We need to either:

1. Use a cluster_id that fits in the 2-bit space (0-3) as the spike source
2. Or increase `number_of_clusters` to accommodate cluster_id 62

But wait — cluster 62 is the **external** cluster (from outside the accelerator — it's the input neuron). The accelerator has 4 clusters internally (0-3). Cluster 62 fires spikes INTO cluster 0. The `incoming_forwarder`

just needs to recognize packet[10:5]=62 and map it to weight row 0.

The `cluster_id` port in `cluster_controller` is used for the `IF_ADDR` opcode to tell the incoming_forwarder which cluster_id to register. The `incoming_forwarder.cluster_id_table` stores it as 7 bits. The comparison in incoming_forwarder.v:

```
if (cluster_id_table[i] != 6'h3F && cluster_id_table[i] == packet[10:5])
```

So `cluster_id_table[i]` holds a **7-bit** value but is compared to 6-bit `packet[10:5]`. The table is `reg [6:0] cluster_id_table[0:31]` initialized to `6'h3F`. Written as `{1'b0, cluster_id[5:0]}`. If `cluster_id` is only 2 bits, we get `{1'b0, 2'b10, 4'bxxxx}` = garbage in the upper bits. Actually no — Verilog zero-extends: `cluster_id[5:0]` where `cluster_id` is 2 bits wide → the upper 4 bits are 0. So `{1'b0, 6'b000010} = 7'h02`.

Then the match check: `cluster_id_table[0] = 7'h02`, `packet[10:5] = 6'h3E`. `7'h02 != 6'h3E` → no match!

The fix: change `number_of_clusters` parameter in the DUT and also update the spike source to use a cluster_id that fits, OR use the correct `number_of_clusters`.

Actually the cleanest fix is to **increase `number_of_clusters` to at least 64** in the DUT instantiation so that `cluster_controller`'s `if_cluster_id` is 6 bits wide. We already have `number_of_clusters = N_CLUSTERS = 4` but we need it to be at least 64 to handle external cluster ID 62.

But wait — this is a design parameter. In the real neuron_accelerator_tb.v, `number_of_clusters=32`. The SNN design uses cluster_id externally. The `incoming_weight_table_rows=64` suggests there can be 64 external clusters.

Looking at cluster_controller.v: `parameter number_of_clusters = 64`. The `if_cluster_id` is `[$clog2(number_of_clusters)-1:0]`. With `number_of_clusters=64`, that's 6 bits, which handles cluster_id 62 correctly.

Fix in testbench: Set `number_of_clusters` to at least 64 in the DUT instantiation. Our testbench has `N_CLUSTERS=4` (internal clusters) but the DUT parameter `number_of_clusters` is used for cluster ID width, not just the count.

Wait — but the DUT actually has 4 clusters internally. The `number_of_clusters` parameter should reflect the MAX cluster_id that can appear in packets. Let me check what `number_of_clusters` controls in neuron_accelerator.v:

In neuron_accelerator.v, `number_of_clusters` is used to:

1. Determine how many clusters to generate (via `cluster_group_count*4`)
2. Pass to each `neuron_cluster` as the cluster ID space

Actually the real cluster count is `cluster_group_count * 4 = CL_GROUPS * 4`. The `number_of_clusters` is just the total pool size for ID addressing. Let me check if changing it from 4 to 64 causes issues:

Looking at the wires in neuron_accelerator.v:

```
wire [32*N_TOTAL-1:0] all_pre_spike;
```

Where `N_TOTAL = number_of_clusters * neurons_per_cluster`. If we change `number_of_clusters` from 4 to 64, `N_TOTAL` becomes 2048 and `all_pre_spike` becomes huge. But we only have 4 actual clusters...

The problem is that `number_of_clusters` in the testbench DUT instantiation is used to size `N_TOTAL`. I need to separate "total clusters in the system (for ID width)" from "actual clusters instantiated".

Looking at neuron_accelerator.v parameters vs actual cluster count: `cluster_group_count` controls the loops, while `number_of_clusters` controls ID width. We need `number_of_clusters >= 64` but `cluster_group_count=1`.

The simplest fix: **change the spike source to use cluster_id=0** (which fits in 2 bits) instead of cluster_id=62. Cluster_id=0 IS one of our actual clusters, but the spike input should come from OUTSIDE. We can use cluster_id=3 (the highest in our 4-cluster system) as the "external" source, then configure incoming_forwarder to recognize it.

Even simpler: **use `number_of_clusters=64` in the DUT** (just like the real system), keep `cluster_group_count=1` so only 4 clusters are actually generated, but fix the wire sizing. This means `N_TOTAL` in the accelerator becomes `64*32=2048`, but `all_pre_spike` becomes huge. The dump FSM then dumps 2048 entries instead of 128.

Let me look at what the real design does — in neuron_accelerator_tb.v, `number_of_clusters=32` and `cluster_group_count=8`, so they match (`8*4=32`). The number of clusters equals the actual cluster count.

The real external input cluster idea: cluster_id in packets comes from OUTSIDE the accelerator — i.e., from a previous layer or input. The `incoming_forwarder` keeps a table of which external cluster IDs map to which weight rows. In the neuron_cluster_tb, `cluster_id=62` was used as the "input neuron" — it's outside the 4-cluster system.

The real fix is to change `number_of_clusters` in the DUT to 64 (matching the parameter space needed for cluster_id 62), and change `N_TOTAL`/`N_SPIKE_W` accordingly. But then the dump FSM writes 64*32=2048 vmem entries. That's too many.

OR: **use cluster_id=3 as the source** (within the 4-cluster system). But cluster 3 is also an internal cluster. However for the testbench, we just need SOME cluster ID that fits in the address width AND is configured in the IF table.

Actually the **cleanest fix**: bump `number_of_clusters` to 64 in the DUT, keep `cluster_group_count=1`. The accelerator will generate only 4 clusters (gen loop runs 1 time for 4 clusters) but the ID space will be 64, giving `N_TOTAL=2048`. Update VMEM_BASE/SPIKE_BASE/N_TOTAL/N_SPIKE_W in the TB.

Wait — but the Dump FSM uses `N_TOTAL` from the DUT parameters to know how many entries to dump. Let me look at the Dump FSM: Now I see the dump FSM uses `N_TOTAL` from the DUT's `localparam`. With

`number_of_clusters=4`, `N_TOTAL=128`, and the dump iterates 128 vmem + 4 spike words = 132 writes per dump. That matches our "264 = 2×132" check.

If we change `number_of_clusters` to 64, `N_TOTAL=2048`, dump becomes 2048+64=2112 writes per dump. That's fine for functionality testing but slows down the sim.

Better fix: Keep `number_of_clusters=4` in the DUT for the testbench, but change the spike source to cluster_id=3 (which fits in 2 bits). Let me update both the init bytes AND the spike injection:

- Change PKT 6: `cluster_id = 3` (fits in 2 bits) instead of 62
- Change Phase 2: inject packet `0x060 = {cluster_id=3, neuron_id=0}` instead of `0x7C0`

Wait — `{cluster_id=3, neuron_id=0}` = `{6'b000011, 5'b00000} = 11'b00001100000 = 0x060`.

But hold on — cluster 3 is one of our internal clusters (gen_neuron_cluster[3])! If we register cluster 3 in cluster 0's incoming_forwarder table, we're saying "when cluster 3 fires, update cluster 0's membrane potentials". That's valid — it's a cross-cluster connection. In the testbench it doesn't matter if it's an "external" cluster or not.

Actually, the cleanest approach matching the real design: **use `number_of_clusters=64` in the DUT, but only check the first 4 clusters' data**. The TB N_TOTAL for checking purposes should match the actual cluster count, but the DUT dump FSM will write 2048 entries. We just check indices 0 and 1.

But writing 2048 entries would make both dumps take much longer (2048+64 cycles each). The simulation will still be fast enough.

Let me go with the simplest fix: **change the cluster_id source from 62 to 3** in the init bytes and spike injection. This avoids changing `number_of_clusters` and keeps the dump small.

Also: the **second issue** is the weight_resolver init — no `[INIT] WR[0] mem[...]` line. Let me think about why.

PKT 9 goes: addr=0x80 (lower router 0 self), count=0x85=133, then `[0x01(sdm port), 0x83(sdm count=131), addr_low=0, addr_high=0, flit_count=0x80=128, 128 bytes]`.

The `self_data_mng` receives: `[0x01, 0x83=131, then 131 bytes of weight_resolver data]`:

- State 0: `port_selected = 0x01[0] = 1` → resolver
- State 1: `count = 0x83 = 131`
- State 2: forwards 131 bytes to resolver

The `weight_resolver` receives `[addr_low=0, addr_high=0, flit_count=0x80=128, then 128 bytes]`:

- State 0: `weight_addr_init[7:0] = 0`
- State 1: `weight_addr_init[15:8] = 0`
- State 2: `weight_flit_count = 0x80 = 128`
- State 3: receives 128 bytes, counter 0..127, at counter=127 `load_weight_mem=1`

But: `weight_resolver` only processes when `chip_mode && load_data`. After the init stream finishes, `network_mode` is set to 0, so `chip_mode=0`. But the bytes are being sent DURING init when

`network_mode=1`. So `chip_mode=1` when processing. ✓

The count mismatch might be the issue: `sdm count=0x83=131` but weight_resolver needs `3 (header) + 128 (data) = 131` bytes. That's correct.

But wait — actually `sdm count=131` means the sdm processes 131 bytes. `weight_resolver` gets `[addr_low(1) + addr_high(1) + flit_count(1) + 128 data] = 131 bytes`. That matches!

Let me think about whether the `sdm count` byte is included in the outer packet count. PKT 9:

- Outer: `[0x80, 0x85, ...]` where `0x85=133`
- The init_router lower receives this: addr=0x80 (self), count=133, then 133 bytes
- The init_router sends 133 bytes to self → `self_data_mng.load_data_in` fires 133 times
- `self_data_mng` gets: `[0x01, 0x83, then 131 bytes]` = exactly 133 bytes ✓

So it should work. Let me check if maybe the weight_resolver `load_weight_mem` never fires because the byte count is wrong. Actually the `weight_resolver` state machine receives `weight_flit_count=128`, then needs exactly 128 bytes in state 3. At `weight_flit_counter == weight_flit_count - 1 = 127`: `load_weight_mem=1`.

But `weight_in_mem[8*weight_flit_counter +: 8] = data` — at counter 127, `weight_in_mem[1023:1016] = data`. That's the last byte.

Actually wait — `weight_in_mem` is `[32*neurons_per_cluster-1:0]`. With `neurons_per_cluster=32`, it's `[1023:0]` = 128 bytes. `weight_flit_count=128` bytes, counter 0..127. At counter=127: `load_weight_mem=1`, writes `weight_mem[0]`. ✓

So the weight_resolver init SHOULD work. Maybe the issue is that when `load_weight_mem=1`, it also transitions `state=0` and changes some signals... Let me add a monitor in the testbench for the weight_resolver state:

Actually, I think the weight_resolver trace would have fired if `load_weight_mem` went high. Since it didn't fire, `load_weight_mem` never went high. This means either the bytes didn't reach the weight_resolver, or the byte count is wrong.

Let me add a monitor for weight_resolver init state to the TB:

```
Made changes.
```

Also fix the cluster_id issue. Change spike source from cluster 62 to cluster 3, which fits in `$clog2(4)=2` bits:

```
Made changes.
```

Now update the spike injection and the expected packet comment:

```
Made changes.
```

Now update the Phase 2 spike injection from `0x7C0` to `0x060`:

```
Made changes.
```

Also update the display message in PHASE 2 description:

```
Made changes.
```

Great progress! Now:

- `[INIT] Cluster0 IF load_cluster_index: cl_id=3` ✓ (fixed!)
- The incoming_forwarder matched and pushed addr=0 ✓
- WR addr 0 pushed ✓
- **But weights are ALL ZERO:** `w[0]=0x00000000 w[1]=0x00000000` ← weight_resolver memory has zeros!

The weight_resolver trace showed it was receiving data (bytes 0..127) but the `[INIT] WR[0] mem[...]` line never appeared. Let me check: The weight_resolver receives all 128 bytes (counter 0..127) but the weights for neuron 0 (bytes 0-3) all show as `0x00`! That's wrong. The expected bytes are:

- byte 0 = `0x00`, byte 1 = `0x00`, byte 2 = `0x05`, byte 3 = `0x00` → weight = 5.0 Q16.16

But from the trace: `ctr=0: 0x00`, `ctr=1: 0x00`, `ctr=2: 0x05`, `ctr=3: 0x00` — wait, looking at the trace again:

```
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=0
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=1
WR load_data: byte=0x05 state=3 flit_cnt=128 ctr=2   ← 5 is here!
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=3
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=4
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=5
WR load_data: byte=0x02 state=3 flit_cnt=128 ctr=6   ← 2 is here!
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=7
```

The bytes `0x05` is at position 2 and `0x02` is at position 6. These correspond to the high byte of the Q16.16 integer part. With little-endian packing:

- `weight_in_mem[7:0] = 0x00` (byte 0 of neuron 0)
- `weight_in_mem[15:8] = 0x00` (byte 1 of neuron 0)
- `weight_in_mem[23:16] = 0x05` (byte 2 of neuron 0) — this IS `0x00050000[23:16]`!
- `weight_in_mem[31:24] = 0x00` (byte 3)

So `weight_in_mem[31:0] = 0x00050000` after bytes 0-3. ✓

But wait — why does the `[INIT] WR[0] mem` line never appear? Because `load_weight_mem` never fired! Let me look at why.

From weight_resolver.v:

```
3: begin
    weight_in_mem[8*weight_flit_counter +: 8] <= data;
    weight_flit_counter <= weight_flit_counter + 1;
    if (weight_flit_counter == weight_flit_count - 1) begin
        state <= 0;
        load_weight_mem <= 1;
    end
end
```

At `ctr=127` (= `weight_flit_count - 1 = 128 - 1 = 127`): `load_weight_mem = 1`. This should fire the `[INIT] WR[0] mem` trace. But it didn't!

Wait — the trace condition is `if (dut.gen_spike_forwarder_4[0].weight_resolver_inst.load_weight_mem)`. And the weight_resolver code:

```
end else begin
    store_selected_buffer <= selected_buffer;
    load_weight_mem <= 0;
end
```

When `chip_mode=0` (after init), `load_weight_mem <= 0`. But when `chip_mode=1 && load_data`, at `ctr=127`: `load_weight_mem = 1` AND simultaneously `state = 0`. BUT the issue might be that `chip_mode` transitions to 0 at the moment ctr=127 is processed!

Looking at the init stream driver:

```
if (init_rom[init_index] !== 8'hxx) begin
    if (ready_in && !load_data_in) begin
        data_in      <= init_rom[init_index];
        load_data_in <= 1;
        ...
    end
end else begin
    load_data_in <= 0;
    if (ready_in && !load_data_in) begin
        start_init   <= 0;
        init_done    <= 1;
        network_mode <= 0; // exit init mode
```

The `network_mode=0` is set when `init_rom[init_index] === 8'hxx` — i.e., AFTER the last byte. The last byte is at `init_index=187`, and `init_rom[188]=8'hxx`. So `network_mode=0` is set on the cycle after the last byte is streamed. At that point `chip_mode=0` and `load_weight_mem=0`. But ctr=127 happens much earlier (@60250) and `network_mode=1` then (@61250 is when init finishes). So `chip_mode=1` during the weight bytes.

But the trace shows `load_weight_mem` never fired! Let me check: maybe the weight_resolver receives the bytes but at ctr=127 the condition `weight_flit_counter == weight_flit_count - 1` doesn't evaluate correctly.

`weight_flit_count = 0x80 = 128`. `weight_flit_count - 1 = 127`. `weight_flit_counter` at the last byte is 127 (after incrementing from 126). Wait — the increment and check happen in the same always block:

```
weight_in_mem[8*weight_flit_counter +: 8] <= data;
weight_flit_counter <= weight_flit_counter + 1;  // registers: takes effect
NEXT cycle
if (weight_flit_counter == weight_flit_count - 1) begin  // checks CURRENT
value
    state <= 0;
    load_weight_mem <= 1;
end
```

At `weight_flit_counter=127`: `8*127=1016`, `weight_in_mem[1023:1016] <= data`. `127 == 128-1` → TRUE → `load_weight_mem <= 1`. But wait:

Looking at the trace again:

```
WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=127 @60250
```

The counter is 127, flit_cnt=128. So `127 == 128-1 = 127` → should set `load_weight_mem=1`. But the trace monitor fires at posedge clk WHEN `load_data=1`. The `load_weight_mem <= 1` takes effect NEXT posedge clk.

So at @60350 (next posedge after @60250): `load_weight_mem` should be 1. Let me check if the trace fires at @60350:

The trace monitor:

```
if (dut.gen_spike_forwarder_4[0].weight_resolver_inst.load_weight_mem)
    $display("[INIT] WR[0] mem[...]");
```

This fires at posedge clk when `load_weight_mem=1`. At @60350: does `load_data` still = 1? And does the `else if(chip_mode && load_data)` branch prevent the `else` branch (which zeros `load_weight_mem`)?

At @60350:

- `chip_mode` = `network_mode` = 1 (still streaming init)
- `load_data` = from weight_resolver input = from `load_data_resolver_4`

But `load_data_resolver_4` comes from `self_data_mng.load_data_out_resolver`. The `self_data_mng` state machine:

```
end else begin
    data_out_forwarder <= 8'b0;
    data_out_resolver <= 8'b0;
    load_data_out_forwarder <= 0;
    load_data_out_resolver <= 0;
end
```

When `load_data_in=0`, both outputs go to 0. So between bytes (when `load_data_in` is not asserted), `load_data_out_resolver=0`. The init stream driver asserts `load_data_in=1` for ONE cycle then waits for `ready_in`. So at @60350, `load_data=0`.

In `weight_resolver`:

```
end else begin
    store_selected_buffer <= selected_buffer;
    load_weight_mem <= 0;   // CLEARS load_weight_mem!
end
```

When `chip_mode=1` but `load_data=0`: falls into `else` → `load_weight_mem <= 0`. So `load_weight_mem` is SET at @60350 but also CLEARED at @60350 (same cycle)? No — the assignments are registered: at @60250 posedge, `load_weight_mem <= 1` is scheduled. At @60350 posedge, the check `chip_mode && load_data` — if `load_data=0`, it takes the else branch and schedules `load_weight_mem <= 0`.

So `load_weight_mem` is 1 only BETWEEN @60250 and @60350 — it's 1 for exactly one posedge-to-posedge period, but our `always @(posedge clk)` trace fires AT @60350 when `load_weight_mem` might already be overridden.

Actually: at the @60350 posedge, `load_weight_mem` = 1 (set from the @60250 FF). The always block READS it (fires `[INIT] WR[0] mem[...]`) AND schedules `load_weight_mem <= 0` (from the else branch). Both happen "simultaneously" at @60350. The $display fires BEFORE the new value takes effect. So the trace SHOULD fire!

But it didn't. Let me check more carefully — maybe `chip_mode=0` at @60350. At @60250, init bytes are still being streamed (`init_done` wasn't set until byte 188). The weight bytes are at @21250..@60250. Byte 188 (the last byte with value 0x00) is at @60250. After that, `init_rom[188]=8'hxx`, so at the next `ready_in` cycle, `network_mode=0` is set.

Wait — how many bytes ARE there in our init ROM? The `load_init_rom` task builds 188 bytes. Let me count:

- PKT 1: 4 bytes (A0 02 00 02)
- PKT 2: 4 bytes (A0 02 10 01)
- PKT 3: 5 bytes (80 03 00 01 02)
- PKT 4: 5 bytes (80 03 00 01 21)
- PKT 5: 5 bytes (00 03 02 00 00)
- PKT 6: 4 bytes (00 02 03 03)
- PKT 7: 13 bytes (00 0B 01 00 08 F9 00 00 03 00 FF F7 01)
- PKT 8: 13 bytes (same, neuron_id=1)
- PKT 9: `1+1+1+1 + 1+1+1 + 128 = 4+131 = 135` bytes

Total: `4+4+5+5+5+4+13+13+135 = 188` bytes ✓

The weight bytes are the last 135 bytes (PKT9), starting from byte `188-135=53`. The weight data starts at byte `53+4(router+count+sdm_port+sdm_cnt) + 3(wr_header) = 53+7 = 60`.

Actually, the ctr values we see:

- ctr=0 starts at @22150. That's byte count since start of weight_resolver data.
- ctr=127 at @60250.

At @60350, is `network_mode` still 1? The init stream driver: after byte 187 (ctr=127 → last byte), `init_index=188`, `init_rom[188]=8'hxx`. The NEXT time `ready_in && !load_data_in` is true:

- `load_data_in=0` (was just de-asserted after byte 187)
- So next cycle, `network_mode=0` is set.

Looking at the waveform timing: if byte 187 is sent at @60250, then `load_data_in` is deasserted at @60350 (next ready cycle). Then the `init_rom[188]=xx` check fires, setting `network_mode=0` and

`init_done=1`. But this is in an `always @(posedge clk)` with `if (ready_in && !load_data_in)`.

The key: when does `network_mode` actually go 0? `network_mode <= 0` is a registered assignment in the init driver `always @(posedge clk)`. So it takes effect ONE cycle after the condition is met.

Actually looking at the init driver again:

```
end else begin
    load_data_in <= 0;
    if (ready_in && !load_data_in) begin
        start_init   <= 0;
        init_done    <= 1;
        network_mode <= 0; // exit init mode
```

Wait — at byte 187: `load_data_in <= 1` (asserted). Next cycle: in the `!= 8'hxx` branch, `ready_in` check: if `ready_in=1` AND `!load_data_in=0` (since `load_data_in` was just asserted), we do nothing (wait). Then the next cycle: `load_data_in` was set to 1 last cycle (registered), so `!load_data_in=0` still. Actually the init driver sets `load_data_in=0` AFTER sending, in the else branch... but only when `ready_in=0` OR we're NOT in the != xx branch.

This is getting complex. Let me just check: `load_weight_mem` at @60350 by adding a direct monitor:

```
Made changes.
```

Now I see the problem clearly:

`load_weight_mem` never went high! Only the reset trace fired: `load_weight_mem changed to 0 @50`. Then ctr reaches 127 at @60250, but `load_weight_mem` NEVER went 1.

Let me look at the weight_resolver logic again VERY carefully:

```
end else if(chip_mode && load_data) begin
    case (state)
        ...
        3: begin
            weight_in_mem[8*weight_flit_counter +: 8] <= data;
            weight_flit_counter <= weight_flit_counter + 1;
            if (weight_flit_counter == weight_flit_count - 1) begin
                state <= 0;
                load_weight_mem <= 1;
            end
        end
```

And at the top of the else-if:

```
end else begin
    store_selected_buffer <= selected_buffer;
    load_weight_mem <= 0;
end
```

The issue: `load_weight_mem` is set to 1 INSIDE the `chip_mode && load_data` branch. But the weight_memory.v uses `load_weight` to write:

```
always @(posedge clk ...) begin
    if (load_weight) begin
        weight_mem[weight_addr] <= weight_in;
    end
end
```

And `weight_addr_in_mem = chip_mode ? weight_addr_init : ...`

At ctr=127: `load_weight_mem` takes effect as 1 at the NEXT posedge (it's registered). At that next posedge, `chip_mode` might be 0 if `network_mode` went to 0.

Let me check: the weight byte ctr=127 arrives at @60250. Then the init driver: after this byte (the 188th byte = last byte), `init_index=188`. At the next `ready_in && !load_data_in` cycle:

- `init_rom[188] = 8'hxx` → else branch: `load_data_in <= 0`, `network_mode <= 0`, `init_done <= 1`

So `network_mode=0` takes effect ONE cycle after the condition is met. The condition `ready_in && !load_data_in` — after byte 187 was loaded (`load_data_in=1`), in the same cycle, the driver has `load_data_in <= 0` in the else branch (since `load_data_in` WAS 1 last cycle but the wire value... wait this is getting confusing).

Let me look at the init driver pattern more carefully:

```
always @(posedge clk) begin
    if (start_init) begin
        if (init_rom[init_index] !== 8'hxx) begin
            if (ready_in && !load_data_in) begin
                data_in      <= init_rom[init_index];
                load_data_in <= 1;
                init_index   <= init_index + 1;
            end else begin
                load_data_in <= 0;
            end
        end else begin
            load_data_in <= 0;
            if (ready_in && !load_data_in) begin
                start_init   <= 0;
```

```
                    init_done    <= 1;
                    network_mode <= 0;
```

The pattern: byte N is sent when `ready_in && !load_data_in`. `load_data_in <= 1` is registered. Next cycle: `load_data_in=1` → `!load_data_in=0` → else branch → `load_data_in <= 0`. Cycle after: `load_data_in=0`, check if `ready_in` → send next byte.

So each byte takes 2 cycles to send (1 asserted, 1 deasserted). At @60250: byte 187 (index 187, last byte) is sent. `init_index <= 188`. At @60350: `load_data_in=1` (still), `!load_data_in=0` → else branch → `load_data_in <= 0`. At @60450: `load_data_in=0`, `init_rom[188]=xx` → else branch: `network_mode <= 0`, `init_done <= 1`. At @60550: `network_mode=0`.

So `network_mode` goes to 0 at @60550. And `load_weight_mem` should go to 1 at @60350 (one cycle after ctr=127 at @60250). At @60350: is `chip_mode=1`? `chip_mode = network_mode` at that point. `network_mode` is still 1 at @60350 (it won't go 0 until @60550). So `chip_mode=1` at @60350. And `load_data` at @60350?

`load_data` = `load_data_resolver_4` = from self_data_mng. The self_data_mng only asserts this when `load_data_in` is high for a byte. At @60350, `load_data_in=1` (still held from byte 187 at @60250). Does the init_router and self_data_mng pipeline have this byte still flowing through?

Actually the path is: `load_data_in → init_router_upper → init_router_lower → self_data_mng → load_data_resolver_4`. Each stage has pipeline delays. The byte sent at @60250 might reach the weight_resolver with a delay of several cycles. Let me trace:

1. @60250: `load_data_in=1`, `data_in=0x00` (byte 187, last weight byte)
2. The init_router_upper processes this byte over a few cycles
3. Eventually self_data_mng receives it and sets `load_data_out_resolver=1`
4. weight_resolver gets `load_data=1`

The weight_resolver trace shows byte 127 at @60250. That means the entire pipeline delay from `load_data_in` to `weight_resolver.load_data` is consistent (all bytes at consistent 300ns intervals = 30 cycles per byte at 100MHz). The LAST byte (ctr=127) arrives at weight_resolver @60250.

Then at @60350 posedge: `weight_resolver` sees `chip_mode=1` (still) and `load_data=0` (byte just finished, self_data_mng would have deasserted `load_data_out_resolver`). So it goes to the `else` branch: `load_weight_mem <= 0`. But ALSO at @60250, `load_weight_mem <= 1` was scheduled (from the `ctr==127` check).

The problem: **at @60350, the `else` branch fires because `load_data=0`, overwriting the `load_weight_mem <= 1` that was scheduled at @60250. The `always @(posedge clk)` block has only ONE driver for `load_weight_mem`, so whichever branch is taken at @60350 determines the output.**

Wait — these assignments are ALL within the SAME `always @(posedge clk)` block. At @60250: `chip_mode=1 && load_data=1` → enters `case(state)=3` → `load_weight_mem <= 1`. At @60350: `chip_mode=1 && load_data=0` → else → `load_weight_mem <= 0`.

So `load_weight_mem` is 1 only at @60350 (the register settles to 1). But at @60350, the always block is EVALUATED and determines the NEXT value. The CURRENT value is 1 at @60350. The `$display` monitor

fires when `load_weight_mem=1` at posedge. BUT the monitor didn't fire.

Wait — could the `always @(dut...load_weight_mem)` sensitivity list not trigger at @60350? In Verilog simulation, `always @(signal)` triggers when the signal CHANGES. If `load_weight_mem` was 0 and changes to 1 at @60350 (from the FF update at @60250), the `always @(...)` SHOULD trigger.

But the output shows `load_weight_mem changed to 0 @50` (reset) and never changes to 1. This means the FF assignment `load_weight_mem <= 1` at @60250 is NOT executing.

Let me look at the weight_resolver.v logic more carefully. The state machine:

```
3: begin
    weight_in_mem[8*weight_flit_counter +: 8] <= data;
    weight_flit_counter <= weight_flit_counter + 1;
    if (weight_flit_counter == weight_flit_count - 1) begin
        state <= 0;
        load_weight_mem <= 1;
    end
end
```

The widths: `weight_flit_count` is `reg [4*$clog2(neurons_per_cluster)-1:0]`. With `neurons_per_cluster=32`, `$clog2(32)=5`, so `4*5=20` bits. `weight_flit_counter` is same width. `weight_flit_count = 0x80 = 128`, so `weight_flit_count - 1 = 127`.

At ctr=127: `weight_flit_counter=127`, `weight_flit_count-1=127`. Should match!

But wait — what is `weight_flit_count - 1`? In Verilog, `128 - 1` where both are 20-bit values → `127`. `127 == 127` → TRUE.

Hmm, but the trace shows the byte at ctr=127 arrives with `state=3` and `flit_cnt=128`. The logic should execute. Let me add an explicit trace for the state=3 completion:

Actually, let me re-examine: the weight_resolver `data` input comes from `data_in_resolver_4`. And `load_data` comes from `load_data_resolver_4`. Both come from `self_data_mng`.

But `data` in weight_resolver has `input [7:0] data`. And `data_in_resolver_4` is from `self_data_mng.data_out_resolver`. Good.

Wait — let me check the actual width of `weight_flit_count` in the weight_resolver instantiation. The DUT uses `neurons_per_cluster=N_PER_CLUSTER=32`. So the width is 20 bits. `weight_flit_count=0x80=128` in 20 bits.

`weight_flit_count - 1`: In Verilog, `weight_flit_count - 1` = 127 in 20 bits.

Actually, I wonder if the issue is that `weight_flit_counter` is only 20 bits but is being compared wrong. Let me check: `weight_flit_count` is set in state 2: `weight_flit_count <= data`. Where `data` is 8 bits. So `weight_flit_count[7:0] = 0x80`, upper bits 0.

And `weight_flit_counter` starts at 0 and increments. At ctr=127: `weight_flit_counter = 127`.

Hmm, but wait — why does the trace show `ctr=127` while the internal counter IS 127? The trace fires `$display` INSIDE the `always @(posedge clk)` block, AT the moment when `weight_flit_counter` is still 127 (before the `weight_flit_counter <= 128` update takes effect). So the condition `weight_flit_counter == weight_flit_count - 1 == 127` IS true at this moment. The `load_weight_mem <= 1` assignment IS made.

But the `always @(dut.weight_resolver_inst.load_weight_mem)` never fires!

This means the signal is NEVER 1. This is very strange. Let me add a direct posedge monitor:

Actually, let me just add it directly in the init_done print: Summarized conversation historyI'll continue from where we left off. The key issue is that `load_weight_mem` never fires in the `weight_resolver`. Let me first read the current state of the relevant files. Found the root cause! The `else` branch at the bottom runs when `chip_mode=0` OR `load_data=0`, and it sets `load_weight_mem <= 0`. When state=3, last flit arrives with `load_data=1`, sets `load_weight_mem <= 1`, but the **next** clock cycle `load_data=0`, so the `else` branch fires and immediately clears it. But since `load_weight_mem` is registered, the write to weight memory needs it high for one cycle — and it IS high for that one cycle (the cycle after the last flit).

Let me check the weight_memory.v to see if it uses the write enable synchronously: Now I have the full picture. Let me run the testbench first to see the actual output: The core problem is clear: the `weight_resolver` NEVER fires `load_weight_mem`. The `[INIT] WR load_weight_mem changed` message only shows at time=50 (reset), never goes to 1. The weight memory has all zeros, so cluster gets zero weights.

Let me analyze the timing issue carefully: Now I see the critical bug. The `else` branch runs when `chip_mode=0` OR `load_data=0`. Since `load_data` is a pulse (only high for one cycle per byte), on every cycle where `load_data=0` (which is most cycles), `load_weight_mem <= 0` gets executed.

At flit 127 (ctr=127): `load_data=1`, `chip_mode=1`, so we enter the `chip_mode && load_data` branch → `load_weight_mem <= 1` is scheduled.

But wait — this is a registered flip-flop. After clock edge where ctr=127, `load_weight_mem` becomes 1. The NEXT cycle, `load_data=0` (no new byte), so the `else` branch runs → `load_weight_mem <= 0`. But `weight_memory` uses `@(posedge clk)` synchronous write, so `load_weight_mem=1` at that same next edge IS visible to `weight_memory`!

Actually, let me re-examine. The logs show WR sees byte 0x00 at ctr=127 at time 60250. The `load_weight_mem <= 1` is scheduled in that same cycle. At time 60350 (next posedge), `load_weight_mem` should be 1, and `weight_memory` should latch. BUT the `else` branch also fires at 60350 (since `load_data=0`), setting `load_weight_mem <= 0`.

In Verilog, blocking vs non-blocking: both are non-blocking (`<=`). At posedge 60350:

- `chip_mode` and `load_data`: if `load_data=0`, the `else` branch runs, so `load_weight_mem <= 0`
- This means `load_weight_mem` was 1 between 60250 and 60350, then goes back to 0

The `weight_memory` write: at posedge 60350, `load_weight_mem` (the input) reads the **current** registered value (i.e., 1 from posedge 60250). So `weight_mem[weight_addr] <= weight_in_mem` SHOULD execute.

Let me add a diagnostic to check what `weight_memory` is seeing: The analysis: `load_weight_mem` is set to 1 at the posedge when ctr==127 (let's call this T1). At T1, `weight_memory` clock edge sees `load_weight=0` (the OLD value before T1). At T2 (next posedge), `load_weight_mem=1` (updated), and `weight_memory` should see `load_weight=1` and write. BUT the `else` branch also fires at T2 (since `load_data=0`), setting `load_weight_mem<=0`. This is a non-blocking assignment so `load_weight_mem` remains 1 until AFTER T2.

So at T2: `weight_memory` reads `load_weight=1` (current value) and writes. The `load_weight_mem <= 0` assignment takes effect AFTER T2. So the write SHOULD happen!

Let me look at a different possibility - what about `weight_in_mem`? At the same T1, `weight_in_mem[8*127 +: 8] <= data` is also a non-blocking assignment. At T2, `weight_in_mem` has the last byte written. Then at T2, `weight_memory` writes `weight_in_mem` into `weight_mem[0]`. BUT at T1, `state <= 0` resets state. Then at T2, the `else` branch runs (load_data=0), which does NOT clear `weight_in_mem`. So `weight_in_mem` still has the full 128-byte weight row at T2.

Wait, but the **init stream driver** continues streaming. After byte 127, it streams the next bytes. The `ready_in` gating... Let me check if `chip_mode` goes to 0 before T2.

Actually, looking at the init sequence: `chip_mode` is `network_mode` in the TB. It stays 1 throughout init. And after init is done, `network_mode <= 0` is set. But the last byte is 127 at time 60250, and `init_done` fires sometime after... Let me check if `chip_mode` drops right after byte 127:

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot: Looking at the output carefully, I can see the critical insight:

```
[INIT] Stream complete at byte 188    ← network_mode set to 0 HERE
[INIT] WR load_data: byte=0x00 state=3 flit_cnt=128 ctr=127 @60250  ← LAST
byte arrives AFTER
```

The `[INIT] Stream complete` message fires (and `network_mode <= 0`) **before** the last byte at ctr=127 arrives at the weight_resolver! This means `chip_mode` drops to 0 while byte 127 is still in transit through the init_router pipeline. So when byte 127 hits the weight_resolver, `chip_mode` is already 0 — the `else` branch runs instead, and `load_weight_mem` never gets set.

Let me verify this by checking the init_router pipeline depth: Now I see the full problem! The `self_data_mng` has the exact same issue as `weight_resolver` — it only processes bytes when `load_data_in` is high, and it passes bytes directly to the resolver one at a time. So the byte pipeline through `init_router → self_data_mng → weight_resolver` works correctly, but the issue is the timing of `chip_mode`.

Let me look at the actual `chip_mode` signal routing: Found it. `.chip_mode(network_mode)` at line 398. The `weight_resolver` uses `chip_mode` both to control the state machine AND in the final condition `if(chip_mode && load_data)`.

The problem: `network_mode` goes to 0 AFTER all init bytes are sent (when `init_done` fires). But the init router pipeline has latency — the last bytes are STILL in transit. The log confirms: `[INIT] Stream complete at byte 188` fires (setting `network_mode=0`) before `ctr=127` arrives at the weight_resolver at time 60250.

Let me check the exact timing: There it is. `Stream complete` fires (setting `network_mode=0`) in the same simulation step as ctr=126's arrival, and ctr=127 comes at 60250 with `chip_mode` already 0.

The fix: **don't turn off `network_mode` until the weight_resolver is also done**. The best approach is to keep `chip_mode` (i.e., `network_mode`) high until `resolvers_done` is asserted — meaning the weight_resolver has finished loading.

The actual fix in the TB: instead of setting `network_mode = 0` immediately in `load_init_rom`, wait for `resolvers_done` (or `accelerator_done`) before setting it. Let me look at the init stream driver: Now I see exactly what to fix. The `network_mode <= 0` is set the moment the last byte is accepted into the router's input — but there's still pipeline latency before the bytes reach the weight_resolver. We need to keep `network_mode = 1` (chip_mode = 1) until the weight_resolver has actually finished loading.

The proper fix has two parts:

1. **In the TB**: Don't drop `network_mode` until `accelerator_done` is asserted (which includes `resolvers_done`).
2. **As a hardware fix**: The `weight_resolver` should latch `load_weight_mem` independently of `chip_mode` once the `state==3` condition triggers — it shouldn't depend on `chip_mode` being high at the commit cycle.

The TB fix is simpler and correct by design — in real hardware, the CPU keeps `chip_mode=1` until it sees a "done" signal from the hardware. Let me fix both — the TB timing and also harden the weight_resolver so it doesn't depend on chip_mode being high at the exact commit cycle: Now I'll apply two fixes:

Fix 1: In the TB init stream driver, separate "bytes sent" from "chip_mode off" — keep `network_mode` high until `accelerator_done` signals all resolvers are done.

Fix 2: Also fix the `weight_resolver` to use a sticky `pending_write` flag so `load_weight_mem` persists correctly independent of `chip_mode`.

Let me do the TB fix first (it's the root cause here):

```
Made changes.
```

Now fix the main sequence to drop `network_mode` after `accelerator_done`:

```
Made changes.
```

But there's another issue: the `accelerator_done` signal includes `resolvers_done` which requires `weight_resolver_done` to be high. The weight_resolver_done logic is:

```
assign weight_resolver_done = ~|selected_buffer && ~|store_selected_buffer;
```

This means "no pending FIFO reads" — which is TRUE during chip_mode init (since no spike packets are sent yet). So `accelerator_done` would be high during init too! Let me check if `resolvers_done` is actually blocking: `accelerator_done` was already high (all FIFOs empty during init), so `network_mode` went to 0 too early again. We need to wait for `accelerator_done` specifically AFTER `init_done`, but the problem is it might already be high. Let me add a monitor to see when `load_weight_mem` fires relative to chip_mode: The problem is clear now: `accelerator_done` fires at ~60000 ns (after `Stream complete`), but byte ctr=127 arrives at 60250 ns — AFTER `accelerator_done` fires and `network_mode` drops to 0. The `accelerator_done` is checking FIFO-level conditions, not whether the router pipeline has fully flushed.

The fundamental issue is that `accelerator_done` doesn't know about in-transit bytes in the init_router pipeline. There are two solid approaches:

Option A (TB-only): After `init_done`, wait for the last byte to reach the resolver by watching `load_weight_mem` go high, or just wait a fixed number of extra cycles for the pipeline to flush.

Option B (Hardware fix): Fix `weight_resolver` to latch `load_weight_mem` as a sticky flag that isn't cleared by the `else` branch — i.e., once state 3 completes its last flit, the write to memory happens on the next clock regardless of `chip_mode`.

Option B is the correct hardware fix. The `weight_resolver` logic has a bug: `load_weight_mem` should be set when the last byte is received and the registered write should happen on the NEXT clock unconditionally. The `else` branch should NOT clear `load_weight_mem` if it was just set: The fix for `weight_resolver`: introduce a `pending_write` flag. When state=3 completes its last flit, set `pending_write=1`. On the NEXT cycle (regardless of `chip_mode` or `load_data`), fire `load_weight_mem=1` and clear `pending_write`. The `else` branch should only clear `load_weight_mem` if `pending_write` is not set:

```
Made changes.
```

Now fix the always block to use `pending_write`:

```
Made changes.
```

Also revert the TB `network_mode` logic back to the original (drop immediately on stream complete), since the hardware is now fixed:

```
Made changes.
```

And revert the main sequence back to the simpler form:

```
Made changes.
```

Now compile and run: `load_weight_mem` still never shows as going high (no `[INIT] WR load_weight_mem changed to 1` message), and no `WR[0] mem written` message. The `pending_write` approach must have an issue. Let me check: Good — byte 127 arrives at 60250 with `chip_mode` still... wait, let me check: `Stream complete` still fires before ctr=127 arrives. Since `pending_write` was supposed to fire on the NEXT cycle regardless of chip_mode, let me check if ctr=127 actually triggered the condition `weight_flit_counter == weight_flit_count - 1`:

The condition is `weight_flit_counter == weight_flit_count - 1`. At ctr=127 (the display shows the current values BEFORE the clock edge), `weight_flit_counter=127` and `weight_flit_count=128`. So `127 == 128-1 = 127` → TRUE. But wait — the monitor `[INIT] WR load_data` fires when `load_data` is high, but it checks `chip_mode` inside? No, the monitor fires unconditionally:

```
always @(posedge clk) begin
    if (dut.gen_spike_forwarder_4[0].weight_resolver_inst.load_data)
        $display(...)
```

So `load_data` is still going high at ctr=127. But `chip_mode` is already 0 (Stream complete happened earlier). The new code is:

```
if (chip_mode && load_data) begin
    case (state)
        3: begin
            ...
```

```
                if (weight_flit_counter == weight_flit_count - 1) begin
                    pending_write <= 1;
```

Since `chip_mode=0` at ctr=127, this branch doesn't execute! The `load_data` monitor fires (because the condition doesn't check chip_mode), but the state machine branch does require `chip_mode`. So `pending_write` never gets set.

The root cause is still the same: `chip_mode` drops to 0 BEFORE byte 127 reaches the weight_resolver. The `pending_write` fix only helps if byte 127 arrives with `chip_mode=1`.

We need to either:

1. **TB fix**: Keep `chip_mode=1` until byte 127 actually reaches the resolver
2. **Hardware fix**: The weight_resolver should process bytes even when `chip_mode` transitions low mid-stream

The cleanest hardware fix: the weight_resolver should use an internal `init_active` flag that goes high when state>0 and stays high until the stream completes, independent of the external `chip_mode`. Let me look at what `chip_mode` actually gates:

In `weight_resolver`, `chip_mode` is used to:

1. Gate the state machine entry: `if(chip_mode && load_data)`
2. Mux the memory address: `chip_mode ? weight_addr_init : ...`

The state machine issue: once in state=3, the resolver is mid-stream. `chip_mode` dropping should NOT abort the stream. We need to track "init_in_progress": The fix: change `if (chip_mode && load_data)` to `if ((chip_mode || state != 0) && load_data)`. This means once the state machine starts (state > 0), it continues processing `load_data` bytes even if `chip_mode` drops. The `chip_mode` is only needed to START a new packet (state=0):

```
Made changes.
```

Also fix the `weight_addr_in_mem` mux — it uses `chip_mode` to select the init address, but `chip_mode` might be 0 when `pending_write` fires. Add `state!=0` as part of the mux:

```
Made changes.
```

Now compile and run: Excellent progress! `load_weight_mem` now fires correctly at 60350 and `mem[0]` is written with `w[0]=0x00050000` and `w[1]=0x00020000`! The weights are in memory. And `all_pre_spike[0] NON-ZERO: 0x00050000` appears at 66150 — just 100ns AFTER the `time_step` pulse at 66050.

The problem now: `all_pre_spike[31:0]=0x00000000` at 66050 (the time_step cycle itself), but goes to `0x00050000` at 66150. The dump FSM is triggering based on `accelerator_done` → then it fires from the time_step. Let me check: `all_pre_spike` becomes non-zero at 66150 but the dump reads it starting right after time_step. Let me check the timing of `dump_done` and what it's reading: Weights are now loaded correctly (`w[0]=0x00050000 w[1]=0x00020000` at @61850). `all_pre_spike` goes to `0x00050000` at 66150 — 100ps (1 cycle) AFTER the `time_step` pulse at 66050.

The issue: the dump FSM is reading `all_pre_spike` too early — it starts at `time_step` rising edge, but `all_pre_spike` is updated ONE cycle LATER (at 66150). The dump reads `all_pre_spike[0]` = 0 (old value).

Let me check the dump FSM timing in neuron_accelerator.v: Now let me see the trigger condition: Found it! The dump FSM triggers on `accelerator_done && !prev_clusters_done` — the rising edge of `accelerator_done`. But at that moment, `all_pre_spike` is being read immediately in `DUMP_VMEM`. The issue is:

1. `time_step` pulse at cycle T (66050ns)
2. Neurons compute accumulate → `all_pre_spike` updates at cycle T+1 (66150ns)
3. But `accelerator_done` might rise at cycle T or T+1 as well
4. The dump FSM reads `all_pre_spike[dump_idx*32 +: 32]` starting from `DUMP_VMEM` — which it enters one cycle after `accelerator_done` rises

Let me check when `accelerator_done` rises relative to `all_pre_spike`: Let me add temporary monitors directly in the TB to track the dump FSM state and what it writes: I need to add a dump FSM timing probe. Let me add it after the existing monitors:

```
Made changes.
```

Now the timing picture is crystal clear:

1. **Dump 2** triggers when `accelerator_done` rises at **61650** (spike pipeline cleared after cluster got weights)
2. During dump 2, `all_pre_spike[31:0]=0x00000000` — because neuron computation hasn't happened yet (no `time_step` issued yet!)
3. `time_step` is issued at **66050** — AFTER dump 2 is already done
4. `all_pre_spike` goes to `0x00050000` at **66150** — AFTER dump 2 is done

The fundamental timing issue: The dump FSM triggers on `accelerator_done` (which means "spike pipeline done"), but the neuron compute (`v_pre_spike`) only happens when `time_step` is pulsed. In the

real use case, `time_step` drives the neurons to compute and THEN `accelerator_done` should be observed. But in this TB, the spike injection happens THEN `accelerator_done` fires, THEN `time_step` is pulsed.

Looking at `neuron_accelerator_tb.v` to understand the intended flow: This is the key! From the reference neuron_accelerator_tb.v (lines 287-294):

```
// Wait for dump to finish BEFORE asserting time_step
// (dump takes 1056 cycles, computation only ~20 cycles)
if (dump_done) begin
    ...
    time_step <= 1;
```

The correct protocol is:

1. Inject spikes into FIFO
2. **Wait for `dump_done`** (the previous timestep's dump finishes)
3. Assert `time_step` to trigger neuron computation
4. The dump FSM then triggers on `accelerator_done` which fires AFTER `time_step` processes (because `time_step` triggers neuron computation, which clears activity, making `accelerator_done` go high)

So the correct order is: **`dump_done` → `time_step` → `accelerator_done` (=new dump trigger)**

In our TB, we have it backwards — we wait for `accelerator_done` THEN pulse `time_step`. The dump captures `all_pre_spike` BEFORE `time_step` runs the neurons.

The fix in the TB: issue `time_step` FIRST (after spike pipeline drains), THEN wait for the dump triggered by the resulting `accelerator_done`. The neurons compute during the `time_step` cycle, then `accelerator_done` rising edge triggers the dump which captures the fresh `v_pre_spike`.

But wait — `v_pre_spike` is captured BEFORE the fire (in the adder), so it's available when the neuron finishes computing after `time_step`. Let me check the neuron pipeline to understand when `v_pre_spike` is stable after `time_step`: Now I understand the full timing:

1. `time_step` rising edge at cycle T → `adder_state` goes to 1
2. Cycle T+1: `v_pre_spike <= weight_added`, `spike` computed, `done <= 1`

So `v_pre_spike` is ready at cycle T+1 (`time_step` + 1 cycle). The dump FSM needs to trigger AFTER cycle T+1.

The current trigger is `accelerator_done && !prev_clusters_done`. After `time_step`:

- Neurons compute → `done` goes high → cluster done → `all_clusters_done` → `accelerator_done` rises

But `accelerator_done` rises ONE cycle after T (at T+1) only when `all_neurons_done` propagates. Let me check when `accelerator_done` rises after `time_step`:

From the trace: `time_step` at 66050, `all_pre_spike` non-zero at 66150 (T+1). The dump FSM needs to start reading at T+1 or later.

The dump FSM current state: triggers on `accelerator_done` rising edge. If `accelerator_done` rises at T+1 (one cycle after `time_step`), then:

- T+1: `accelerator_done` rises, dump FSM enters `DUMP_VMEM`, reads `all_pre_spike[0*32+:32]` (idx=0), sets `portb_we=0` (first cycle), `dump_idx` increments
- T+2: `portb_we=1`, writes `all_pre_spike[0]` (which was latched at T+1) — BUT `all_pre_spike` changes at T+1 too!

Actually the non-blocking assignment means: at T+1, both `v_pre_spike <= weight_added` fires AND the dump FSM reads `all_pre_spike[0]`. The question is whether `all_pre_spike` is the OLD value (0) or NEW value (0x00050000) at T+1.

Since `v_pre_spike` is registered with non-blocking assignment, it updates at the END of clock edge T+1. The combinational `all_pre_spike` wire reads the register. At T+1 posedge:

- The register `v_pre_spike` still holds the OLD value (0) when the dump FSM samples it
- Then at the END of T+1, `v_pre_spike` updates to `0x00050000`

So the dump FSM reading at T+1 sees the OLD value! It needs to read at T+2 (or later).

BUT wait — from the trace, `accelerator_done` rises at 61650 (when the spike pipeline clears after weights are loaded), which is BEFORE `time_step` at 66050. That's the second `accelerator_done` rise during inference (spike pipeline drains). Then there's a third `accelerator_done` rise after `time_step` when neurons compute.

Looking at the trace again:

```
accelerator_done ROSE @61650  ← spike pipeline cleared
accelerator_done ROSE @61950  ← (another glitch?)
time_step @66050
all_pre_spike NON-ZERO @66150
```

The dump FSM triggers on `accelerator_done` at 61650 — that's the WRONG trigger. At that point, `time_step` hasn't been pulsed yet, so neurons have zero `v_pre_spike`.

The real design intent from neuron_accelerator_tb.v is:

1. `dump_done` (from previous dump) → `time_step` assertion
2. `time_step` → neurons compute → `all_neurons_done` → `accelerator_done` rises (third rise)
3. This third `accelerator_done` rise → dump FSM triggers → reads correct `v_pre_spike`

The issue in our TB: we assert `time_step` AFTER `accelerator_done` has already risen (at 61650). So we're triggering the dump at the wrong time.

The correct TB pattern from the reference: **wait for `dump_done` (from previous timestep's dump), THEN assert `time_step`**. The dump FSM will then trigger on the NEXT `accelerator_done` rise which follows neuron computation.

Let me fix the TB sequence to match the reference pattern: I need to fundamentally restructure Phase 2. The correct pattern from the reference TB is:

1. Wait for first `dump_done` (already done in Phase 1)
2. Inject spike(s)
3. Assert `time_step` (which triggers neuron compute)
4. Wait for the `dump_done` that comes AFTER neuron compute finishes

The key insight: `time_step` should be asserted WHILE (or just after) the spike pipeline has drained, not after waiting for `accelerator_done`. The spike accumulator adds into `weight_added` which `v_pre_spike` captures. The accumulation happens via `load_weight_in` driving the accumulator — that's separate from `time_step`.

Let me re-read the neuron cluster flow more carefully: Now I understand the full picture:

The **accumulator** works as follows:

- `load` (from weights arriving): adds `weight_in` to `accumulated_reg` every cycle it's high
- On `time_step` rising edge: `accumulated_out <= accumulated_reg` (captures accumulated sum), then `accumulated_reg <= 0` (resets)

So `accumulated_out` only updates ON the `time_step` edge. Then `potential_adder` uses `accumulated_out` (as `input_weight`? No wait...). Let me check the full neuron flow: Now I understand the full data flow. The `potential_adder`'s `input_weight` is `accumulated_out` (inference mode). The accumulator accumulates weights during spike processing, then on `time_step` rising edge: `accumulated_out <= accumulated_reg`. Then on the NEXT cycle (`adder_state == 1`), `v_pre_spike <= weight_added` where `weight_added = decayed_potential + input_weight (= accumulated_out)`.

The timing:

- T: `time_step` rises → `accumulated_out <= accumulated_reg` and `adder_state <= 1`
- T+1: `v_pre_spike <= weight_added` (decayed + accumulated)

So `v_pre_spike` is stable at T+1. For the dump FSM to read the correct value, it needs to start reading at T+1 or later.

But the current dump trigger is `accelerator_done` rising edge. After `time_step`:

- T: `time_step` rises, neuron starts computing
- T+1: `adder_done <= 1` → `all_neurons_done` → changes `accelerator_done`

The dump reads `all_pre_spike` at the same T+1 when both it becomes non-zero AND the dump FSM transitions to DUMP_VMEM. Since this is the same clock edge, the dump FSM in `DUMP_IDLE` sees `accelerator_done` rise and enters `DUMP_VMEM`. In `DUMP_VMEM` (at T+1), `portb_we=0` (first cycle has no write), `portb_din <= all_pre_spike[0]`. Then at T+2, `portb_we=1` and it writes.

But at T+1, what does the dump FSM see for `all_pre_spike[0]`? Since `v_pre_spike` is non-blocking (`<=`), it still holds the OLD value when sampled at posedge T+1! The new value takes effect AFTER T+1.

So the dump reads `all_pre_spike[0]` at T+1 (old=0) and writes it at T+2, getting the WRONG value.

Wait — let me reconsider. Looking at the dump FSM logic:

```
DUMP_VMEM: begin
    portb_we  <= 1'b1;
    portb_en  <= 1'b1;
    portb_addr <= vmem_base_addr + dump_idx;
    portb_din  <= all_pre_spike[dump_idx*32 +: 32];
    if (dump_idx == N_TOTAL - 1) begin
        dump_state <= DUMP_SPIKES;
        dump_idx   <= 11'd0;
    end else
        dump_idx <= dump_idx + 1'd1;
end
```

`portb_din <= all_pre_spike[dump_idx*32 +: 32]` is a non-blocking assignment. At posedge T+1 (when dump enters DUMP_VMEM), it samples the CURRENT value of `all_pre_spike` which is the OLD value (0). The new `v_pre_spike` value only appears after T+1 settles.

At posedge T+2: `portb_we` is now 1, `portb_din` has the old value (0), and `all_pre_spike` has the NEW value. The dump reads `all_pre_spike` again for dump_idx=1, but writes the stale value for dump_idx=0.

So even if the dump triggers at the right time, it's always reading one cycle behind. The dump needs to start ONE cycle LATER. The simplest fix is to add a 1-cycle delay in the dump FSM trigger.

But actually — looking at the dump_idx=0 trace from the debug output:

```
state=1 idx=0 portb_we=0 portb_din=0x00000000 @61750  ← portb_we=0 means
idx=0 write hasn't happened yet
state=1 idx=1 portb_we=1 portb_din=0x00000000 @61850  ← writing idx=0 value
from 61750
```

At 61750 (T+1 after accelerator_done rise at 61650+100 = 61750), dump_idx=0, portb_we=0, portb_din=0. This is the FIRST cycle of DUMP_VMEM - it samples `all_pre_spike[0]` which is 0 (no time_step yet). At 61850 (T+2), dump_idx=1, portb_we=1 — writing the stale 0 for neuron 0.

The actual fix: after `time_step`, the dump FSM should wait for ONE more cycle before capturing `all_pre_spike`. This is cleanest done by modifying the dump trigger to add a 1-cycle pipeline delay.

However, looking at the **reference TB pattern** again — `dump_done` → `time_step`. This means `time_step` fires AFTER the previous dump completes. Then the NEXT `accelerator_done` rise (after neuron compute) triggers a NEW dump.

In the reference TB, the flow for each timestep is:

1. Spikes are already injected (from previous step or current)
2. `dump_done` fires (previous dump done)
3. `time_step` is pulsed → neurons compute → `accumulated_reg` → `accumulated_out`, `v_pre_spike` computed

4. `accelerator_done` rises (neurons done + pipeline empty)
5. New dump starts, capturing `v_pre_spike` which was computed in step 3

The timing works in the reference because: after `time_step` at T, neurons output `v_pre_spike` at T+1. The `accelerator_done` rising edge happens at T+1 (when `all_neurons_done` goes high). The dump FSM enters DUMP_VMEM at T+1, samples `all_pre_spike[0]` — but this is **also** T+1, so it's sampling the non-blocking result that hasn't been committed yet!

There must be something I'm missing. Let me check `all_neurons_done`:

From neuron_accelerator.v line 158: `wire all_neurons_done = &all_neurons_done_vec;`

And `accelerator_done` = `all_clusters_done & all_forwarders_done & main_fifo_empty_in & main_fifo_empty_out & resolvers_done`.

`all_clusters_done` derives from the cluster done signals. Let me check the neuron_cluster to see how `done` propagates: So `cluster_done = incoming_forwarder_done & outgoing_enc_done & spikes_done` and `neurons_done_out = spikes_done`. The `spikes_done` is what the neuron_layer calls `neurons_done` (which is `&neuron_done_reg`).

`neuron_done_reg` is set when each neuron's `adder_done` fires. `adder_done` fires at T+2 (one cycle after `time_step` rising edge triggers `adder_state=1`, then next cycle `done <= 1`).

So the timeline:

- T: `time_step` rises
- T+1: `adder_state` = 1, accumulator outputs `accumulated_out`
- T+2: `v_pre_spike <= weight_added`, `done <= 1` (adder_done)
- T+2: `neurons_done` = 1, `spikes_done` = 1, `cluster_done` = 1, `all_clusters_done` = 1
- T+2: `accelerator_done` rises (assuming forwarders/FIFOs already done)

So `accelerator_done` rises at T+2, and `v_pre_spike` also updates at T+2 (same clock edge). When the dump FSM samples at T+2:

- `v_pre_spike` register holds T+1 value (old=0) at posedge T+2 (before non-blocking updates)
- Dump FSM enters DUMP_VMEM and reads `all_pre_spike[0]` = 0

At T+3:

- `v_pre_spike` now holds new value (0x00050000)
- Dump FSM writes `portb_din` (which was assigned at T+2) = 0, idx=0

So there's always a 1-cycle read delay. The dump FSM reads `all_pre_spike` ONE cycle before the new value is stable.

The correct fix is to add a 1-cycle pipeline register in the dump FSM between detecting `accelerator_done` and actually starting to read `all_pre_spike`. This way:

- T+2: `accelerator_done` rises → dump FSM records "trigger seen" but waits one more cycle
- T+3: dump FSM enters DUMP_VMEM, reads `all_pre_spike[0]` = 0x00050000 ✓

Let me add this to the dump FSM in neuron_accelerator.v: The fix: add a 1-cycle delay between the `accelerator_done` rising edge and starting to read `all_pre_spike`. The cleanest approach is to add an intermediate `DUMP_WAIT` state that holds for one cycle, then transitions to `DUMP_VMEM`. This gives the registered `v_pre_spike` values time to settle.

Actually, re-reading the code: the issue is that on the cycle when `accelerator_done` rises (T+2), the dump FSM transitions from `DUMP_IDLE` to `DUMP_VMEM` AND `v_pre_spike` also updates. Since both happen at the same clock edge with non-blocking semantics, the dump FSM reads the old (pre-update) value of `all_pre_spike`.

The fix: add a `DUMP_LATCH` state between `DUMP_IDLE` and `DUMP_VMEM` that waits one cycle. During this cycle, `v_pre_spike` has settled. Then `DUMP_VMEM` starts reading at T+3.

I'll also latch `all_pre_spike` into a register to avoid potential glitches during the dump:

```
Made changes.
```

Now compile and run: Still failing! The dump FSM still reads `all_pre_spike=0x00000000` at 61850 (the latch cycle). This is the SECOND dump (at 61650), which fires because `accelerator_done` rose when the spike pipeline cleared — BEFORE `time_step` was issued at 66050. So `all_pre_spike` is still 0 at 61850.

The issue is that we have 2 dumps firing before `time_step`: one at 1150 (init) and one at 61650 (spike pipeline cleared). The REAL inference dump would be a THIRD dump, triggered AFTER `time_step` fires at 66050. But the TB is only waiting for 2 `dump_done` pulses.

The trace shows `all_pre_spike` goes to 0x00050000 at 66150, but the dump 2 happened at 61650-62xxx. There's no third dump because the TB ends after dump 2!

The flow is:

1. `accelerator_done` rises @1150 → dump 1 (all zeros, init)
2. Spike injected, propagates through pipeline
3. `accelerator_done` rises @61650 → dump 2 starts (all zeros, no `time_step` yet)
4. Dump 2 finishes
5. `time_step` asserted @66050 → neurons compute → `v_pre_spike` updates @66150
6. `accelerator_done` rises again (3rd time) → dump 3 would fire — but TB doesn't wait for it!

The TB checks `dump_number == 2` and exits. We need to check `dump_number == 3` and verify the THIRD dump.

But we need to restructure: the spike should be injected AFTER the first dump, and `time_step` should be issued when `accelerator_done` is stable (spike pipeline cleared), then wait for the NEXT `accelerator_done` (from neuron compute) → dump 2 with real values.

Actually the simplest fix is to follow the reference TB pattern exactly:

1. Phase 1: Init. Wait for first `dump_done`.
2. Phase 2: Inject spike. Assert `time_step` immediately. Wait for dump_done.

But we need to make sure the accumulator has the weights before `time_step`. The accumulator adds `neuron_weight_in` each cycle `load_data_in` (which here is `acc_load`) is high. This means the weight arrives via `load_weight_in` at the cluster and `acc_load = !chip_mode && load_data` (in inference mode).

The spike flow: spike → incoming_forwarder → push addr to weight_resolver → weight_resolver pops → weights → cluster `load_weight_in=1` → `acc_load=1` → accumulator adds weight for ALL 32 neurons simultaneously (one cycle).

So after the spike packet fully propagates (by the time `accelerator_done` rises at 61650), the accumulator has already accumulated the weights for neuron 0 (5.0) and neuron 1 (2.0). `time_step` can then be asserted.

The correct TB flow should be:

1. Init, wait for dump 1
2. Inject spike, wait for pipeline drain (`accelerator_done` stable at 61650)
3. Issue `time_step` → neurons compute, `v_pre_spike` latched
4. Wait for dump 3 (triggered by `accelerator_done` after neuron compute)

The third `accelerator_done` rise happens after T+2 from `time_step`. Let me check: after `time_step` at 66050:

- 66150: `adder_done=1` → `neurons_done=1` → `all_clusters_done=1` → `accelerator_done` rises (3rd time)

So there WILL be a 3rd `accelerator_done` rise, leading to dump 3 with real values. The TB just needs to wait for that third dump.

Let me update the TB to:

1. Inject spike
2. Wait for second `dump_done` (dump 2 = spike pipeline cleared, still zeros)
3. Issue `time_step`
4. Wait for third `dump_done` (dump 3 = real values)
5. Verify dump 3

The check becomes `dump_number == 3`:

```
Made changes.
```

Now update the verification section to check dump 3 and change the expected `dump_number` to 3: Now update the verification to check 3 dumps instead of 2:

```
Made changes.
```

```
Made changes.
```

Now compile and run: Getting closer! `all_pre_spike` goes to 0x00050000 at 75350 (after `time_step` at 75250). But `dump_number` stays at 2 — there's no third `accelerator_done` rise after `time_step`. The timeout fires.

The problem: after `time_step`, `accelerator_done` doesn't rise again. This is because the neuron's `done` signal doesn't propagate to `accelerator_done` correctly in this setup, or `all_neurons_done` is not part of the `accelerator_done` computation here.

Let me check: `accelerator_done = all_clusters_done & all_forwarders_done & main_fifo_empty_in & main_fifo_empty_out & resolvers_done`. There's no `all_neurons_done` in this! The `all_clusters_done` uses `cluster_done`, and:

```
assign cluster_done = incoming_forwarder_done & outgoing_enc_done &
spikes_done;
```

`spikes_done = neurons_done_out` which is `neuron_done_reg` (from `neuron_layer`). So `cluster_done` does include `spikes_done`. But `spikes_done` goes HIGH when neurons are done — then `cluster_done` goes HIGH — then `all_clusters_done` goes HIGH — then `accelerator_done` should rise.

BUT: `cluster_done` = `incoming_forwarder_done & outgoing_enc_done & spikes_done`. After `time_step`, `spikes_done` goes high, but do `incoming_forwarder_done` and `outgoing_enc_done` also stay high? They should already be high (no spike traffic). Let me verify what prevents `accelerator_done` from rising after `time_step`: `accelerator_done` never rises a third time. The `spikes_done` signal after `time_step` — let me check the `neuron_layer` done logic: Found it! In neuron_layer.v:

```
else if (time_step && !prev_time_step) begin
    neuron_done_reg <= 0;  // ← CLEARS done reg
    spikes_out <= 0;
end
```

On the `time_step` rising edge, `neuron_done_reg` is cleared to 0. This means `neurons_done = 0` → `spikes_done = 0` → `cluster_done` drops → `all_clusters_done` drops → `accelerator_done` drops.

Then when the neurons finish (T+2), `neuron_done[j]` fires → `neuron_done_reg[j] <= 1` → `neurons_done` goes back to 1 → `cluster_done` → `all_clusters_done` → `accelerator_done` rises again.

But looking at the initialization: on reset, `neuron_done_reg[j] <= 1` for ALL j! So at startup, `neurons_done=1`, `cluster_done=1`, `accelerator_done=1` immediately. That's why the first dump happens right at @1150.

After `time_step` pulses: `neuron_done_reg <= 0` (cleared). Then neurons compute: `neuron_done[j]` fires and `neuron_done_reg[j] <= 1`. Then `neurons_done=1`, `cluster_done=1`, `accelerator_done` rises.

BUT wait — the probe shows `accelerator_done` only rises at @61650 (after spike pipeline drain). Does it ALSO rise after `time_step` (neuron compute)?

Let me add a more targeted probe for `accelerator_done` after `time_step`: Let me add the probe directly into the TB:

```
Made changes.
```

Excellent — now I can see exactly what's happening. After `time_step` at 75250:

- 75350: neurons compute → `all_pre_spike` goes to 0x00050000
- 75450: `all_cl=0` (cluster_done drops — likely due to outgoing encoder being triggered), `neurons_done=0` (because `neuron_done_reg` was cleared by `time_step`)
- 75550: `neurons_done=1` (neuron done reg set back), `all_cl=0` still
- 75850: `all_cl=1` but `all_fwd=0`
- **76450**: `all_fwd=1` but `fifo_eo=0` — **main FIFO out is NOT empty!**

The spike from neuron 0 is being forwarded back out through the outgoing encoder and sitting in the main output FIFO! The `main_fifo_empty_out=0` prevents `accelerator_done` from going high.

This makes sense: neuron 0 fires → spike is encoded by outgoing_enc → goes into outgoing forwarder → SF4 → SF8 → main FIFO out. The main FIFO out stays non-empty because nobody is reading it (`main_fifo_rd_en_out=0`).

The TB needs to drain the output FIFO after `time_step`. Let me check — in the reference neuron_accelerator_tb.v, does it drain the output FIFO? `assign main_fifo_rd_en_out = ~main_fifo_empty_out;` — in the reference TB, the output FIFO is always being drained! Any spike that gets forwarded out is immediately consumed. This ensures `main_fifo_empty_out=1` for `accelerator_done` to go high.

The simple fix in our TB: set `main_fifo_rd_en_out = 1` always (drain the output FIFO):

Continue to iterate? Copilot has been working on this problem for a while. It can continue to iterate, or you can send a new message to refine your prompt.

ravindu439: @agent Continue: "Continue to iterate?"

GitHub Copilot:

```
Made changes.
```

Now compile and run with the output FIFO drain fix: `[DUMP2] NON-ZERO V_MEM neuron=0 = 0x00050000` and `neuron=1 = 0x00020000` are appearing! The `accelerator_done` rises at 76550 (after the spike is drained from the output FIFO). The dump FSM is latching at 76750 (1 cycle after the rise). Now let me check if the 3rd dump finishes and the checks pass: **7/7 PASS!** All tests are passing. Let me get the clean final output: **7/7 PASS!** Now let me clean up the verbose PROBE monitors from the TB since they're no longer needed for debugging, and verify the L2 tests still pass:

```
Made changes.
```

Also remove the dump FSM internal state probe (only needed for debug):

```
Made changes.
```

Now do a clean final run and also verify the L2 neuron_cluster_tb still passes: Now verify the L2 test still passes: L2 still passes 8/8. Now let me create the promised GTKWave script with labeled signal groups:

Made changes.

Now let's do a final check of all three modified hardware files to make sure they're clean: All three levels pass. Let me print a final summary of every file that was changed: All 7 tests pass. Here's a summary of everything that was fixed in this session:

Summary of Changes Made

Bug 1: `weight_resolver` — `load_weight_mem` never fired (root cause)

File: weight_resolver.v

Problem: `chip_mode` (driven by `network_mode`) drops to 0 the moment the last init byte enters the `init_router`. But the router pipeline has ~10 cycles of latency — byte 127 (the last weight flit) arrives at the `weight_resolver` **after** `chip_mode` is already 0. The old `if(chip_mode && load_data)` guard blocked the final flit.

Fix: Two-part:

1. Added a `pending_write` flag — set when flit 127 arrives in state=3, committed to `load_weight_mem=1` on the next cycle **unconditionally** (not gated by `chip_mode`).
2. Changed the state-machine guard to `if ((chip_mode || state != 0) && load_data)` — once a packet has started (state>0), it completes even if `chip_mode` drops mid-stream.

Bug 2: Dump FSM — reading `all_pre_spike` 1 cycle too early

File: neuron_accelerator.v

Problem: `accelerator_done` rises at the same clock edge that `v_pre_spike` registers are updated (both are non-blocking assignments). The dump FSM entered `DUMP_VMEM` and read `all_pre_spike` at that same edge — sampling the **old** (pre-update) value.

Fix: Added a `dump_latch_cycle` one-cycle pipeline delay inside `DUMP_IDLE`. When `accelerator_done` rises, the FSM sets `dump_latch_cycle=1` but stays in `DUMP_IDLE` for one more cycle. The next cycle it latches both `all_pre_spike` and `all_spikes` into stable registers (`latched_pre_spike`, `latched_spikes`), then enters `DUMP_VMEM` which reads from those latches.

Bug 3: Output FIFO blocking `accelerator_done` after `time_step`

File: known_value_dump_tb.v

Problem: When neuron 0 fires, the spike is forwarded back out through `outgoing_enc → SF4 → SF8 → main_fifo_out`. With `main_fifo_rd_en_out=0`, this FIFO never drained, keeping `main_fifo_empty_out=0` and `accelerator_done=0` permanently.

Fix: Changed `reg main_fifo_rd_en_out = 0` to `wire main_fifo_rd_en_out = ~main_fifo_empty_out` — always drain the output FIFO, exactly matching the reference neuron_accelerator_tb.v.

Correct 3-dump Protocol

The correct dump sequence (now documented in the TB) is:

1. **Dump 1** — `accelerator_done` after init (all zeros, init state)
2. **Dump 2** — `accelerator_done` after spike pipeline drains (accumulator loaded, no `time_step` yet)

3. **`time_step` → neuron compute → Dump 3** — `accelerator_done` after neurons compute, `latched_pre_spike` has correct Q16.16 values

ravindu439: so can i know what was the changes that made in entire neuron acclearator for this dumping purpose for the v mem and the spikes . becuase this inference accelarator from a previous group that make onchip inference only SoC so we take that to implement our SoC s forwar pass part . and now i want to know what are the changes that be for this purpose like state machine and those things . and also tell me why that part how do we reduce that number of clock cycles like in risc v processors we use forwarding kind of mechanisms right so can we use those kind of things ?

GitHub Copilot: Summarized conversation historyI'll look at all the modified files to give you a complete picture of every change made. Summarized conversation historyI'll read the key modified files to give you an accurate, complete answer. Now I have everything I need to give you a thorough, technically precise answer to both questions.

Question 1: Full explanation of all changes made to the original accelerator for the dump purpose

The original accelerator was purely inference-only: it computed spikes and updated membrane potentials (`final_potential`), but exposed nothing to the CPU for backpropagation. The changes form a clean **datapath from synapse → neuron register → bus → FSM → shared memory**.

Change 1 — potential_adder.v: Capture `v_pre_spike` (the backprop-critical value)

Why this location? The `potential_adder` is the single point in the pipeline where `weight_added = decayed_potential + input_weight` exists as a combinational signal, *before* the fire-and-reset decision is applied. This is exactly the quantity the surrogate gradient needs: the membrane potential just before the threshold comparison, at the moment the spike decision is made.

What was added:

```
output reg [31:0] v_pre_spike,   // NEW port

// inside adder_state == 1 (the compute step):
v_pre_spike <= weight_added;     // latch pre-fire value
spike       <= (weight_added > v_threshold);
final_potential <= reset_value;  // post-fire (with reset applied)
```

Why `adder_state==1`? The original FSM waits one cycle after `time_step` rises before computing (to let `weight_added` settle from the combinational adder). `adder_state==1` is therefore the first stable cycle. Latching here gives `v_pre_spike = weight_added` at the same moment `spike` and `final_potential` are decided — all three outputs are consistent and simultaneous.

Holds until next timestep — the `else` branch does not touch `v_pre_spike`, so it retains its value through the entire inter-timestep idle period. This gives the Dump FSM plenty of time to read it.

The same pattern was applied to the `neuron_int_all` variant (where the equivalent latch point is `adder_state==2`).

Change 2 — Port propagation up the hierarchy (`neuron → neuron_layer → neuron_cluster → neuron_accelerator`)

Each level adds a `v_pre_spike_out` bus that is simply the concatenation of all children's outputs. No logic — pure wiring. This is the standard "bus aggregation" pattern for parametric designs.

Level	Added port	Width
`neuron.v`	`v_pre_spike_out[31:0]`	32 bits per neuron
`neuron_layer.v`	`v_pre_spike_out[32*N-1:0]`	32 bits × neurons_per_cluster
`neuron_cluster.v`	`v_pre_spike_out[32*N-1:0]`	same
neuron_accelerator.v	`all_pre_spike[32*N_TOTAL-1:0]`	32 bits × all neurons

In neuron_accelerator.v the `generate` loop that instantiates clusters also wires each cluster's slice of `all_pre_spike`:

```
.v_pre_spike_out(all_pre_spike[(i*4+j)*32*neurons_per_cluster +:
32*neurons_per_cluster])
```

Similarly `all_spikes[N_TOTAL-1:0]` and `all_vmem` were collected for spike dumping and debug respectively.

Change 3 — Dump FSM in neuron_accelerator.v

This is the core new block. It converts the wide parallel buses (`all_pre_spike`, `all_spikes`) into a sequential stream of 32-bit writes to shared memory via Port B (a standard dual-port BRAM interface).

New ports added to the accelerator:

- `portb_addr[15:0]`, `portb_din[31:0]`, `portb_we`, `portb_en` — the BRAM Port B write interface
- `dump_done` — 1-cycle pulse when done; CPU can poll or interrupt on this
- `vmem_base_addr[15:0]`, `spike_base_addr[15:0]` — CPU-configured layout in shared memory
- `current_timestep[3:0]` — tells the FSM which timestep row to write into

FSM states and their roles:

```
DUMP_IDLE  →  (dump_latch_cycle flag, 1 extra cycle)  →  DUMP_VMEM  →
DUMP_SPIKES  →  DUMP_DONE  →  DUMP_IDLE
```

State	What it does

State	What it does
`DUMP_IDLE`	Watches for rising edge of `accelerator_done`. On the trigger cycle, sets `dump_latch_cycle=1` and captures `t_latch = current_timestep`. Does NOT read `all_pre_spike` yet (see pipeline delay below). On the *next* cycle, latches `all_pre_spike → latched_pre_spike` and `all_spikes → latched_spikes`, then transitions to DUMP_VMEM.
`DUMP_VMEM`	Writes one `latched_pre_spike[idx*32+:32]` word per cycle. Address = `vmem_base_addr + t_latch*N_TOTAL + idx`. Runs for exactly `N_TOTAL` cycles.
`DUMP_SPIKES`	Writes one 32-spike-packed word per cycle from `latched_spikes`. Address = `spike_base_addr + t_latch*N_SPIKE_W + idx`. Runs for `N_SPIKE_W = ⌈N_TOTAL/32⌉` cycles.
`DUMP_DONE`	Deasserts `portb_we/en`, returns to IDLE. `dump_done` pulses high here (combinational decode of state).

The 1-cycle `dump_latch_cycle` pipeline delay — why it exists:
`accelerator_done` is an AND of many registered signals. On the rising edge of `accelerator_done`, the non-blocking assignments that produced `v_pre_spike` inside the neurons are in the *same* Verilog delta cycle. Reading `all_pre_spike` in that same `posedge clk` would read the *old* value (the one from the previous timestep). Waiting one extra cycle allows all `v_pre_spike <= weight_added` register updates to fully commit, so `latched_pre_spike` captures the correct new values.

Why latch at all (why not read `all_pre_spike` directly during DUMP_VMEM)?
Because `all_pre_spike` is a combinational wire driven by registered outputs in the neurons. During DUMP_VMEM and DUMP_SPIKES, the system is idle (no `time_step`), so the values are stable — you *could* read directly. However latching:

1. Protects against any glitch or mid-dump `rst_potential`
2. Makes the FSM's Port B writes depend only on registers, never combinational paths → better timing closure at high frequency
3. Gives a clean snapshot semantics matching what a DMA-like block would do

---

Change 4 — weight_resolver.v bug fix (prerequisite for `accelerator_done` to ever go high)

This was not a "dump feature" per se, but without it, `accelerator_done` never asserted, so the Dump FSM never triggered.

Root cause:
The `weight_resolver` receives weight init packets from the CPU via a byte-serial init router. The router has ~10 cycle pipeline latency. The CPU sets `chip_mode=1` (= `network_mode`) to signal "send init data", then clears it. By the time the *last* byte of the last weight packet exits the router pipeline, `chip_mode` has already dropped to 0.

The original guard `if (chip_mode && load_data)` blocked that last byte. As a result, `load_weight_mem` never fired for the last neuron's weight row, leaving the weight memory incomplete and the weight resolver's `done` signal stuck low — keeping `accelerator_done` stuck low permanently.

Three-part fix:

1. **`pending_write` flag:** When the last flit of a packet arrives (`weight_flit_counter == weight_flit_count - 1`), instead of immediately asserting `load_weight_mem=1` (which would be overwritten to 0 by the `else` branch one cycle later before chip_mode drops), set `pending_write=1`. On the very next clock cycle, `pending_write` unconditionally asserts `load_weight_mem=1` then clears itself.

2. **`state != 0` guard:** Changed `if (chip_mode && load_data)` to `if ((chip_mode || state != 0) && load_data)`. Once a packet is mid-stream (`state > 0`), it completes regardless of `chip_mode`. This handles the case where `chip_mode` drops *while bytes are still in the router pipeline*.

3. **Address mux extension:** The combinational mux that selects `weight_addr_in_mem` was gated by `chip_mode`. Extended to `(chip_mode || pending_write || state != 0)` so the init address is held stable during both the mid-stream states and the one-cycle `pending_write` commit.

Change 5 — Testbench FIFO drain fix (testbench correction, not RTL)

In `known_value_dump_tb.v`, the output FIFO read-enable was initially hardwired to `0`. When neuron 0 fired, its spike was re-encoded and pushed into `main_fifo_out`. With `rd_en=0`, the FIFO never emptied, keeping `main_fifo_empty_out=0`, which kept `accelerator_done=0` indefinitely. The fix was:

```
// Before (stuck):
reg main_fifo_rd_en_out = 0;

// After (always drain):
wire main_fifo_rd_en_out = ~main_fifo_empty_out;
```

This matches the reference `neuron_accelerator_tb.v`. In a real system the CPU (or a downstream accelerator) would read the FIFO; the testbench just auto-drains it.

Question 2: Can clock cycle count be reduced with forwarding-like mechanisms?

Yes — and the analogy to RISC-V forwarding is surprisingly direct. Here is a systematic analysis.

Where do the extra cycles come from?

The total dump latency per timestep is:

```
1 cycle   (dump_latch_cycle — pipeline delay to let v_pre_spike registers
settle)
+ N_TOTAL cycles  (DUMP_VMEM — one word per cycle)
+ N_SPIKE_W cycles (DUMP_SPIKES — one word per cycle)
+ 1 cycle  (DUMP_DONE)
```

For `N_TOTAL=128`: 1 + 128 + 4 + 1 = **134 cycles** per timestep dump.
For `N_TOTAL=2048` (64 clusters × 32 neurons): 1 + 2048 + 64 + 1 = **2114 cycles**.

The sequential bottleneck is that shared memory's Port B is 32-bit wide and can only accept **one word per cycle**. That fundamental constraint cannot be bypassed with any amount of forwarding — it is a structural hazard, not a data hazard.

What RISC-V forwarding actually solves, and the analogy here

In a RISC-V pipeline, forwarding eliminates **RAW (Read-After-Write) data hazards**: a later instruction that needs a result can receive it from an intermediate pipeline register rather than waiting for it to be written to the register file and read back. It does **not** increase throughput — it removes *unnecessary* stalls.

In our case, the only stall is the `dump_latch_cycle` — a 1-cycle bubble inserted because `v_pre_spike` is being written by the neurons in the same rising edge that triggers the dump. This is exactly the RAW hazard pattern:

```
Cycle N:    v_pre_spike <= weight_added   (write in neuron registers)
Cycle N:    accelerator_done rises         (trigger)
Cycle N+1:  latch all_pre_spike           (safe read — registers have
settled)
Cycle N+2:  begin DUMP_VMEM
```

The forwarding equivalent would be: **read `weight_added` directly** (the combinational wire, not the register) at the moment the trigger fires. Then you can skip the 1-cycle delay:

```
// Instead of: latched_pre_spike <= all_pre_spike  (registers, 1 cycle
late)
// Use a separate wire from the combinational adder output:
wire [32*N_TOTAL-1:0] all_pre_spike_comb;  // directly from weight_added
// Latch immediately on trigger cycle:
latched_pre_spike <= all_pre_spike_comb;
```

This saves **1 cycle** — analogous to bypassing the WB stage in a RISC-V pipeline. The tradeoff is longer combinational paths (the latch now captures a wire that travels from inside every adder all the way to the accelerator top level), which may hurt timing closure at high clock rates. At typical FPGA frequencies (50–100 MHz for this design) it is likely acceptable.

Other cycle-reduction opportunities (non-forwarding)

Technique	Savings	Complexity	Notes
Eliminate `dump_latch_cycle` (forwarding as above)	1 cycle / dump	Low	Safe if timing allows the long combinational path

Technique	Savings	Complexity	Notes
Wider Port B (64-bit or 128-bit)	2× or 4× DUMP_VMEM	Medium	Requires matching BRAM configuration
Overlap VMEM and SPIKES writes (two address ports)	Eliminates N_SPIKE_W	High	Needs a second BRAM port or separate spike SRAM
Start dump before all clusters finish (pipelined)	Up to `cluster_latency` cycles	High	Requires per-cluster done tracking + partial latch
DMA burst mode (write N words in a burst)	Reduces address overhead	Medium	Depends on BRAM/AXI interface support
Skip spike dump if spike_count==0	N_SPIKE_W cycles	Low	Add a `|all_spikes` check before entering DUMP_SPIKES

The last one is essentially free: most timesteps in a sparse SNN have very few spikes. If `all_spikes == 0`, skip DUMP_SPIKES entirely — saving 4–64 cycles depending on N_TOTAL. Add this to the transition from DUMP_VMEM:

```
// at end of DUMP_VMEM, instead of always going to DUMP_SPIKES:
if (dump_idx == N_TOTAL - 1) begin
    dump_state <= (|latched_spikes) ? DUMP_SPIKES : DUMP_DONE;
    dump_idx   <= 11'd0;
end
```

Summary

Question	Answer
Can forwarding reduce cycles?	Yes — 1 cycle saved by reading the combinational `weight_added` wire directly instead of the registered `v_pre_spike`, eliminating the `dump_latch_cycle` bubble
Is this the dominant cost?	No — the dominant cost is the N_TOTAL sequential writes (structural hazard on the 32-bit port), which no forwarding can remove
Best practical win	Skip DUMP_SPIKES when `all_spikes==0` (free, ~4–64 cycles) + widen Port B to 64-bit (halves DUMP_VMEM, moderate effort)
Forwarding saves the whole dump?	No — for N_TOTAL=128, forwarding saves 1/134 cycles (0.7%). The port width is the real bottleneck